

Ingegneria del Software

NECKER

26 Settembre 2026

Indice

1	Ingegneria, qualità e processi	8
2	Modelli di ciclo di vita del software	13
2.1	Modelli sequenziali	13
2.1.1	Modello a cascata	13
2.1.2	Modello a V (denti di pesce cane)	14
2.2	Modelli iterativi	14
2.2.1	Modello a cascata con singola retroazione	14
2.2.2	Modello prototipale	14
2.3	Modelli incrementali	15
2.3.1	Modello a fontana	15
2.3.2	Problemi comuni dei modelli incrementali	15
2.3.3	Pinball Life-Cycle	15
2.4	Modelli trasformativazionali	16
2.5	Metamodello a spirale	16
2.5.1	Modello COTS (Component Off The Shelf)	16
2.6	Metodologie Agile	17
2.6.1	Manifesto Agile	17
2.6.2	Lean Software	17
2.6.3	Kanban	18
2.6.4	Scrum	18
2.6.5	Crystal	18
3	Extreme Programming (XP)	18
3.1	Test Driven Development (TDD)	18
3.2	Fondamenti di XP	19
3.3	Principi XP	20
3.4	Figure e responsabilità	21

3.5	Tecniche XP (13 tecniche principali)	21
3.6	Planning Game — Dettagli	22
3.6.1	Tecniche di stima	22
3.6.2	Velocity	23
3.7	Brevi cicli di rilascio	23
3.8	Uso di una metafora	23
3.9	Semplicità di progetto	24
3.10	Testing	24
3.11	Refactoring	24
3.12	Pair Programming (programmazione a coppie)	25
3.13	Proprietà collettiva del codice	26
3.14	Integrazione continua	26
3.15	Settimana di 40 ore	26
3.16	Cliente sul posto	27
3.17	Standard di codifica	27
3.18	They're just rules	27
4	Software Configuration Management (SCM)	37
4.1	Storia	37
4.2	Manufatti e configurazioni	37
4.3	Centralizzato vs decentralizzato	38
4.4	Meccanismo di base	38
4.5	Repository	38
4.6	Cosa tracciare	39
4.7	Accesso concorrente	39
4.8	SCM (Software Configuration Management) distribuito – vantaggi e limiti	39
4.9	Git	40
4.9.1	<code>git commit</code> - record changes to the repository	40
4.9.2	<code>git switch</code> - switch branches	40
4.9.3	<code>git merge</code> - unire rami di sviluppo	41
4.9.4	<code>git reset</code> - reset current HEAD	41
5	Git Workflow e Strumenti	42
5.1	Branch in Git	42
5.2	GitFlow	42
5.2.1	Feature Branch	42
5.2.2	Release Branch	43
5.2.3	Hotfix Branch	43
5.3	Limiti di Git e GitFlow	43
5.4	<code>git request-pull</code>	43

5.5	Hosting Centralizzato	43
5.6	Pull Request / Review	44
5.7	Gerrit	44
5.8	Strumenti Open Source	44
5.8.1	Build Automation	45
5.8.2	Bug Tracking	45
5.9	Unified Process (UP / RUP)	45
5.10	Refactoring e principi di design	47
6	Conoscenze Preliminari	48
6.0.1	Linguaggio Object Orientation	48
6.0.2	Principi SOLID	48
6.0.3	Reference Escaping (errore da non fare)	48
6.0.4	Encapsulation e Information Hiding	49
6.0.5	Legacy e Deprecated	49
6.0.6	Immutabilità	49
6.0.7	Code Smell	49
6.0.8	Tell-Don't-Ask	50
6.0.9	Interface Segregation (approcci pratici)	50
6.1	Esempio: gerarchia Card / Deck (up front)	50
7	Design Patterns	53
7.1	Definizione generale	53
7.2	Categorie dei pattern (GoF)	53
7.3	Meta-patterns	53
7.4	Singleton	54
7.5	Iterator	56
7.6	Chain of Responsibility	56
7.7	Flyweight	58
7.8	Null Object	58
7.9	Strategy (Delegation)	59
7.10	Observer	60
7.11	Adapter	62
7.11.1	Class Adapter	62
7.11.2	Object Adapter	63
7.11.3	Confronto Class vs Object Adapter	63
7.12	Model View Controller (MVC)	77
7.13	Model View Presenter (MVP)	81
7.14	Builder	84
7.14.1	Telescoping Constructor Pattern	84
7.14.2	JavaBeans Pattern	84

7.14.3	Builder Pattern	84
8	UML	87
8.1	Analisi del testo naturale	87
8.1.1	Organizzazione del design	87
8.1.2	Noun Extraction	87
8.1.3	Criteri di sfoltimento	87
8.1.4	Relazioni tra classi	88
8.1.5	Stato: concreto vs astratto	88
8.1.6	Casi particolari	88
8.2	State Diagram	88
8.2.1	Concetto e struttura	88
8.2.2	Esempio: flip-flop di Moore	89
8.2.3	Automi di Mealy	89
8.3	In UML	89
8.3.1	Azioni e eventi	89
8.3.2	Guardie	89
8.3.3	Eventi speciali	90
8.4	Superstate	90
8.5	Class Diagram	90
8.5.1	Concetto e struttura	90
8.5.2	Regole grafiche	90
8.5.3	Relazioni UML	91
8.6	Sequence Diagram	91
8.6.1	Concetto e struttura	91
8.7	Activity Diagram	91
8.7.1	Concetto e struttura	91
8.7.2	Livelli di astrazione	91
8.7.3	Sincronizzazione	92
8.7.4	Decisioni	92
8.7.5	Swim Lane	92
8.8	Use Case Diagram	92
8.8.1	Concetto e struttura	92
8.8.2	Componenti	92
8.8.3	Relazioni	92
8.9	Component Diagram	93
8.9.1	Concetto e struttura	93
8.9.2	Identificazione componenti	93
8.10	Deployment Diagram	93

9	Mocking	95
9.0.1	nomenclatura:	95
9.1	Struttura di un Test	95
9.2	Test Double	95
9.3	Regole per il Mocking	96
9.4	Mockito	96
9.5	Esempio: Pattern Observer (Pull)	97
9.6	Injection con Mockito	98
9.7	Mocked Construction	98
9.8	Mocking di Iterable	99
9.9	Riepilogo Tipi di Test Double	100
10	Verifica e Convalida	101
10.1	Verifica e Convalida	101
10.1.1	Terminologia	101
10.1.2	Malfunzionamento (Failure)	101
10.1.3	Difetto (Fault/Anomalia)	102
10.1.4	Sbaglio (Mistake)	102
10.1.5	Caso Ariane 5	102
10.2	Tecniche di Verifica e Convalida	102
10.2.1	Classificazione generale	102
10.2.2	Piramide concettuale	103
10.2.3	Metodi Formali	103
10.2.4	Testing	103
10.2.5	Debugging	104
10.3	Testing	104
10.3.1	Definizioni	104
10.4	Legenda	104
10.4.1	Proprietà	105
10.4.2	Utilità di un test	106
10.4.3	Criteri di selezione	106
10.4.4	Implicazioni tra criteri di copertura	109
10.4.5	Altri criteri (cicli)	109
10.4.6	Mappa finale delle implicazioni	110
11	Analisi Statica	111
11.1	Definizione generale	111
11.2	Principali strumenti	111
11.3	Compilatori	111
11.4	Analisi Data Flow	112
11.5	Esempio pratico	112

11.6	Sequenze e cammini	113
11.7	Espressioni regolari	113
11.8	Testing e analisi statica	113
11.9	Terminologia	114
11.10	Criteri di copertura derivati	114
11.11	Oltre le variabili	115
11.12	Criterio di copertura del budget	115
12	Tecniche di Review	116
12.1	Introduzione	116
12.2	Principali tecniche	116
12.3	Bebugging	116
12.4	Analisi Mutazionale	117
12.4.1	Criterio di copertura dei mutanti	117
12.4.2	Generazione dei mutanti	117
12.4.3	High Order Mutation (HOM)	118
12.4.4	Automazione	118
12.5	Object-Oriented Testing	118
12.5.1	Ereditarietà e collegamento dinamico	118
12.5.2	Testing di una classe	118
12.5.3	Criteri di copertura per classi	119
12.6	Testing Funzionale	119
12.6.1	Tecniche principali	119
12.6.2	Testing delle interfacce	120
12.6.3	Classi di equivalenza	120
12.6.4	Test di frontiera	120
12.6.5	Category Partition	120
12.6.6	Conclusioni	121
13	Object Orientation e Testing Funzionale	121
14	Software Inspection	122
14.1	Fagan Code Inspection	122
14.1.1	Checklist	123
14.2	Automazione	123
14.3	Pro e Contro	124
15	Confronto tra Tecniche di Verifica e Convalida	124
15.1	Combinare le Tecniche	124
16	Gruppi di Test Autonomi	125

17 Modelli Statistici	125
18 Debugging	126
18.1 Tecniche di Debugging	126
18.2 Automazione del Debugging	127

1 Ingegneria, qualità e processi

Storia

- Anni '50-'60: necessità di superare metodi "artigianali" di sviluppo software.
- Problemi principali: comunicazione tra cliente e programmatore, dominio applicativo non più solo matematico.
- Anni '70: nascita di metodi, processi e strumenti per migliorare qualità software, approccio ingegneristico.

Approccio ingegneristico

- Target: obiettivo da raggiungere.
- Metric: metrica per misurare il raggiungimento del target.
- Method/Process/Tool: tecniche per avvicinarsi all'obiettivo.
- Measurements: esperimenti controllati per valutare l'efficacia:
 - Risultati soddisfacenti $\Rightarrow$ accettare metodi/processi.
 - Risultati insoddisfacenti $\Rightarrow$ rivedere target, metrica, processi o metodi.
- Tipi di obiettivi:
 - Risolvere problemi nello sviluppo.
 - Assicurare qualità del software.

Problemi principali

- Comunicazione:
 - Sviluppatore $\leftrightarrow$ Cliente: domini diversi, più persone coinvolte.
 - Sviluppatore $\leftrightarrow$ Sviluppatore: stili e specializzazioni differenti.
- Dimensioni software: milioni di righe di codice, migliaia di anni-uomo (se un dipendente lavora per 8 ore al giorno allora 8 ore = un giorno uomo).
- Malleabilità software: versioni, evoluzioni, variazioni di target.

- Misura anni-uomo: non indicativa, non lineare con numero di sviluppatori.

Qualità

qualità che vogliamo che il software abbia

- Tipi:
 - Esterne: colte dai clienti.
 - Interne: colte dagli sviluppatori, influenzano anche qualità esterne.
- Requisiti vs Specifiche:
 - Requisiti: ciò che il cliente vuole, mutevoli.
 - Specifiche: interpretazione dei requisiti da parte del programmatore, base contrattuale.

Qualità del software

- Correttezza: software rispetta specifiche.
- Affidabilità: ci si può fidare del funzionamento rispetto ai requisiti.
- Robustezza/Safety: comportamento accettabile anche fuori specifiche.
- Usabilità: facile e intuitivo per utenti, test qualitativi e quantitativi.
- Prestazioni/Efficienza: performance esterna vs uso interno risorse.
- Verificabilità: proprietà facilmente verificabili, correlata a leggibilità o metodi formali.
- Riutilizzabilità: componenti riutilizzabili, aumenta affidabilità e verificabilità.
- Manutenibilità:
 - Riparabilità: correzione difetti.
 - Evolvibilità: aggiunta nuove funzionalità.
- Perfettibilità: migliorare qualità senza cambiare funzionalità.

Leggi rilevanti

- Glass (G1): errori nei requisiti $\Rightarrow$ fallimento progetti.
- Nielsen-Norman (NN23): usabilità misurabile.
- McIlroy (MI15): riuso software aumenta produttività e qualità.
- Lehman (L27-28): sistemi evolvono, complessità aumenta se non gestita.

Debito tecnico

- Rimandare correzioni crea “debito tecnico”.
- Il debito crea interessi: aumento difficoltà e tempo di sviluppo di nuove funzionalità a causa del brutto codice.

Qualità del processo

trattate le qualità del software che vogliamo, ora dobbiamo introdurre le qualità del processo, cioè come lo facciamo, un buon processo deve avere:

- Robustezza: resistere imprevisti. (es: mancanza improvvisa di personale)
- Produttività: (tenere conto che la produttività di un team è meno della produttività sommata dei singoli.)
- Tempismo:
 - Rispetto scadenze.
 - Gestione volatilità requisiti.
 - Cogliere il contesto di mercato.

Processo di produzione software

- Problemi:
 - Requisiti cambiano.
 - Software $\neq$ scrivere solo codice.
 - Comunicazione complessa.

- Necessità di rigore (Ipotesi Bauer-Zemanek: Metodi formali riducono in maniera significativa gli errori di progettazione, oppure permettono di eliminarli e risolverli prima.).
- Organizzazione attività:
 - Identificare attività.
 - Definire precedenze e responsabilità.
- Inizialmente si usava il modello code-and-fix, molto inefficace.

Fasi del ciclo di vita

- **Studio di fattibilità:** scenari, architetture, costi, ROI (return of investment).
- **Analisi e specifica requisiti:**
 - Comprendere dominio, dialogo con cliente, identificare stakeholder.
 - Definire funzionalità (cosa non come).
 - Dizionario comune, requisiti non funzionali. (bisogna essere molto precisi)
 - Output dell'analisi: documento di specifica, piano di test, manuale utente. il documento può essere di tipo: descrittivo o operativo (legge di David: il valore dei modelli che rappresentano il software da diversi punti di vista dipendono dal punto di vista preso (assunto), ma non c'è nessuna vista che è la migliore per ogni scopo.)

modello descrittivo vs operativo

- descrittivo: modello che descrive come è fatto, più una documentazione specifica
- operativo: una sorta di prototipo, più concentrato sul: deve essere più o meno così
- **Progettazione (design):** il secondo step è la progettazione e design:
 - Architettura software.
 - Scomposizione in moduli/oggetti.

- Identificazione pattern.
- **Programmazione e test unità:** sviluppo moduli, test unitari, stub e driver.
- **Integrazione e test di sistema:** integrazione moduli, test incrementale, alpha testing.
- **Consegna, installazione e manutenzione:** beta testing, deployment, manutenzione correttiva/adattiva/perfettiva.
- **Altre attività:**
 - Documentazione.
 - Verifica e controllo qualità.
 - Gestione processo e configurazioni.

2 Modelli di ciclo di vita del software

Non esiste un modello universalmente valido.

Tipologie di modelli:

- **Prescrittivi:** impongono regole precise.
- **Descrittivi:** descrivono processi esistenti senza imporli.

2.1 Modelli sequenziali

2.1.1 Modello a cascata

Caratteristiche:

- Fasi in sequenza: Requisiti $\rightarrow$ Progetto $\rightarrow$ Codifica $\rightarrow$ Testing $\rightarrow$ Prodotto.
- Ogni fase produce un semilavorato (document-based), una volta passati alla fase successiva si considera il lavoro per quella fase finito.
- Separazione dei compiti, rigore e pianificazione temporale semplice.

Pro:

- Document-based.
- Buona suddivisione dei compiti.
- Pianificazione chiara.

Contro:

- Rigidità, impossibilità di tornare indietro.
- Congelamento dei sottoprodotti.
- Monoliticità, assenza di manutenzione reale.

Nota storica: Non elogiato, ma usato come paragone negativo da Royce (1970), che ne criticava i limiti proponendo approcci incrementali.

2.1.2 Modello a V (denti di pesce cane)

Idea: Variante del modello a cascata con fasi di: *verifica* e *validazione*.

Verifica: controllo formale rispetto alle specifiche (no cliente).

Validazione: controllo rispetto alle esigenze del cliente con feedback continuo.

Obiettivo: Coinvolgere il cliente e garantire coerenza in ogni fase.

2.2 Modelli iterativi

Idea generale: permettere la ripetizione di fasi per aumentare flessibilità e adattabilità ai cambiamenti.

2.2.1 Modello a cascata con singola retroazione

- Consente un salto indietro di una sola fase (es. da Testing a Codifica).
- In realtà notiamo che abbiamo la possibilità di fare iterazioni complete fino alla consegna dato che tornati indietro di uno possiamo valutare di tornare ancora indietro di uno.
- Difficile pianificare tempi e avanzamento.

2.2.2 Modello prototipale

Caratteristiche:

- Introduce prototipi usa e getta (throw-away).
- Scopo: ricevere feedback o testare idee/strumenti.
- **Prototipi pubblici:** per chiarire requisiti cliente.
- **Prototipi privati (spike):** esplorativi, per comprendere meglio il problema.

Legge di Boehm (L3): La prototipizzazione riduce significativamente gli errori di analisi dei requisiti e di design, specialmente per le interfacce utente.

Iteratività: ogni ciclo produce e distrugge un prototipo, fino alla versione definitiva.

2.3 Modelli incrementali

Definizione: modelli iterativi in cui ogni iterazione include anche una consegna. **Differenze:**

- **Iterativo:** iterazioni su sviluppo.
- **Incrementale:** iterazioni su tutte le fasi, incluse specifiche e rilascio.

Vantaggi: integrazione naturale della manutenzione e gestione evolutiva del software.

2.3.1 Modello a fontana

- Nascita: 1993, opposto al modello a cascata.
- Si può sempre tornare alla fase iniziale (software pool: insieme di parti di codice riutilizzabili) per correggere errori.
- Introduce manutenzione ed evoluzione post-consegna.
- Perdita di linearità e impossibilità di stimare tempi.

2.3.2 Problemi comuni dei modelli incrementali

- Pianificazione complessa, stato di avanzamento meno visibile.
- Necessità di esperti sempre disponibili.
- Possibile non convergenza del progetto.
- Overlapping tra iterazioni (quindi due versioni prodotte nello stesso momento) → poco tempo per feedback.

2.3.3 Pinball Life-Cycle

- Modello creato come critica ironica di Ambler: ordine delle attività casuale e non controllabile.
- Processo non misurabile e caotico (pessimistico ma realistico in contesti non strutturati).

2.4 Modelli trasformatzionali

- Opposti del Pinball: massima formalità e controllo.
- Partenza da requisiti informali, trasformazioni formali fino al codice finale.
- Ogni passo produce un prototipo formalmente corretto.
- Introduce prove formali di correttezza e versioning tramite trasformazioni.

Applicazioni: a causa della loro estrema rigidità, usati prevalentemente in ambienti di ricerca o progetti hardware/software (es. processori).

2.5 Metamodello a spirale

Caratteristiche:

- Metamodello incrementale basato sui rischi (*risk-based*), attenzione, non è un vero e proprio modello incrementale poichè si differenzia in questo modo.
- Fasi:
 1. Determinazione di obiettivi, alternative e vincoli.
 2. Valutazione alternative e rischi.
 3. Sviluppo e verifica.
 4. Pianificazione iterazione successiva.
- Raggio della spirale = costi crescenti.

Variante Win-Win: considerazione dei rischi contrattuali, ricerca di equilibrio cliente-sviluppatore, infatti ad ogni iterazione si cerca di trovare un compromesso tra cliente e sviluppatori.

2.5.1 Modello COTS (Component Off The Shelf)

Idea: sviluppo basato su componenti preesistenti (interni o commerciali).

Fasi:

1. Analisi dei requisiti.
2. Analisi dei componenti disponibili.

3. Modifica dei requisiti (se necessario).
4. Progetto del sistema col riuso.
5. Sviluppo e integrazione.
6. Verifica del sistema.

Pro: riuso, velocità.

Contro: complessità di integrazione, inefficienza per funzionalità inutili.

2.6 Metodologie Agile

Idea: metodologie PRESCRITTIVE nate “dal basso” per risolvere i limiti dei metodi tradizionali.

2.6.1 Manifesto Agile

> implica: più importante

- Individui e collaborazione > processi e strumenti.
- Software funzionante > documentazione.
- Collaborazione col cliente > contratto.
- Risposta al cambiamento > seguire un piano.

Business model: pagamento a tempo di sviluppo con collaborazione diretta col cliente.

2.6.2 Lean Software

- Ispirato al *Lean Manufacturing* di Toyota.
- Obiettivo: ridurre sprechi, evitare prodotti non consegnabili (si cerca di evitare di fare demo o prototipi che farebbero solo sprecare tempo dato che poi verrebbero buttati via).
- Riutilizzare codice, parallelizzare processi, posticipare scelte vincolanti.
- Investire nel benessere degli sviluppatori.

2.6.3 Kanban

- Minimizzare il *work in progress* (concentrarsi solo su una cosa e non fare cose in parallelo).
- Tabelle con 5 colonne: backlog, da fare, in esecuzione, in testing, fatto.
- Ogni card assegnata a uno sviluppatore o coppia.
- Se un membro è bloccato, aiuta gli altri invece di iniziare nuovo lavoro.

2.6.4 Scrum

- Iterazioni brevi (2-4 settimane).
- Requisiti fissi durante l'iterazione (sempre per mantenere lo sviluppatore su un singolo lavoro).
- Cliente coinvolto solo tra le iterazioni.
- Focus e stabilità per il team.

2.6.5 Crystal

- Introduce la **comunicazione osmotica**: conoscenza condivisa tra membri, ogni fase è curata da tutti, non esiste un esperto che è l'unico responsabile di una singola fase.
- Codice di proprietà collettiva → robustezza.
- Efficace per team piccoli (minori di 10 membri).
- per team grandi esistono evoluzioni, come il modello "SAFE".

3 Extreme Programming (XP)

sempre una metodologia agile

3.1 Test Driven Development (TDD)

- Tecnica di **progettazione** software che fa emergere il design più semplice possibile.
- Non è né verifica né scrittura del codice, ma un **approccio alla scrittura**.

- Formula: **TDD** = **test-first** + **baby steps**.
- Obiettivo: **feedback rapido** e continuo.
- Ciclo principale: **Rosso** → **Verde** → **Refactoring**.
 - **Rosso**: scrivo il test prima del codice → fallisce (feature non esiste ancora).
 - **Verde**: scrivo il minimo codice per far passare il test.
 - **Refactoring**: miglioro il codice mantenendo i test verdi.
- Ciclo ripetuto ogni 2–10 minuti ⇒ focus su piccoli compiti.
- Vantaggi:
 - Progetta codice **testabile e semplice**.
 - Definisce interfacce chiare e riduce le dipendenze.

3.2 Fondamenti di XP

- Creato da **Kent Beck** (fine anni '90, progetto Chrysler).
- Variabili fondamentali (dipendenti, l'una dall'altra):
 - **Portata**: funzionalità da implementare (variabile principale).
 - **Tempo**: risorse temporali disponibili.
 - **Qualità**: deve restare costante (il software deve funzionare).
 - **Costo**: risorse economiche/personale impiegate.
- In XP: qualità è costante, si bilanciano costo e tempo → **portata è adattiva**, (rendere la portata adattiva permette di rendere costanti le altre variabili).
- Approccio incrementale e consegna continua:
 - Cliente parte del team e riceve **release incrementali**.
 - Può modificare requisiti e priorità in corso d'opera.
- Risultato: rapporto più **collaborativo e trasparente** tra cliente e sviluppatore.

3.3 Principi XP

- Riprende principi classici:
 - separazione interessi: separare i compiti
 - astrazione e modularità: usare la giusta astrazione, giusto linguaggio ecc
 - Anticipazione del cambiamento: oin futuro il progetto può evolvere, meglio prevenire che curare
 - generalità: meglio interfacce piu generali per far fronte a un possibile cambiamento
 - rigore: rigidità e precisione nelle specifiche
 - incrementalità: non tutto insieme ma uno sviluppo solido un po' alla volta
- Introduce nuovi principi, ereditandone solo alcuni di quelli classici (separazione di interessi data per scontata):
 - **Feedback rapido**: continuo da test, colleghi e cliente (es. standup meeting).
 - **Presumere semplicità**: evitare complessità inutile nel codice e nei ruoli.
 - **Accettare il cambiamento**: prevedere modifiche ma senza progettarle in anticipo.
 - **Modifica incrementale**: iterazioni brevi e funzionalità piccole.
 - **Lavoro di qualità**: focus sul benessere dello sviluppatore.
- Contrasto chiave: **Semplicità vs Anticipazione del cambiamento**.
 - XP: meglio codice semplice, facilmente modificabile.
 - Curve dei costi:
 - * **Bohem (1976)**: costo modifiche esponenziale, secondo lui non pianificare implica maggiore difficoltà in futuro.
 - * **XP**: secondo XP invece il costo è logaritmico grazie a refactoring e linguaggi moderni.

3.4 Figure e responsabilità

- Figure principali:
 - **Cliente**: conosce il dominio, definisce funzionalità e priorità.
 - **Sviluppatore**: implementa, stima tempi e definisce dettagli tecnici.
 - **Manager**: supervisiona, pianifica e monitora.
 - **Tracker**: monitora problemi e metriche del team (es. issue aperti).
- Concetti chiave:
 - **Business value**: misura l'importanza di una feature rispetto al costo.
 - Trasparenza e fiducia tra cliente e sviluppatore.

3.5 Tecniche XP (13 tecniche principali)

1. **Planning Game**: pianificazione iniziale di ogni iterazione.
2. **Brevi cicli di rilascio**: release ogni ~ 2 settimane.
3. **Uso di una metafora**: linguaggio comune per sviluppatori e cliente.
4. **Presumere la semplicità**: codice chiaro e diretto.
5. **Testing**: test continui e automatizzati.
6. **Refactoring**: miglioramento continuo del codice.
7. **Pair Programming**: due sviluppatori per una postazione.
8. **Proprietà collettiva**: il codice appartiene a tutto il team.
9. **Integrazione continua**: build e test frequenti.
10. **Settimana da 40 ore**: evitare sovraccarico.
11. **Cliente sul posto**: collaborazione diretta e quotidiana.
12. **Standard di codifica**: uniformità nello stile.
13. **They're just rules**: flessibilità nell'applicazione.

3.6 Planning Game — Dettagli

- Scopo: determinare quali feature implementare nella prossima iterazione.
- Strumento: **user stories** scritte su carte (soggetto, azione, motivazione).
- Carte contengono: ID, descrizione, test d'accettazione, valore di business.
- Fasi:
 1. Il cliente scrive le carte (user stories).
 2. Il team stima i tempi (in ore, giorni o story points).
 3. Il manager seleziona le carte da realizzare in base a valore e tempo.
- Problemi di stima:
 - Stime molto diverse → carta ambigua.
 - Stime alte → story troppo grande (fare **splitting**).
 - Effetto **àncora**: la prima stima influenza le successive.

3.6.1 Tecniche di stima

- **Planning Poker**:
 - Ogni membro sceglie una carta coperta (numeri da 0 a 100, scala non lineare).
 - Le carte vengono scoperte insieme.
 - Chi ha scelto valori estremi spiega la propria scelta.
 - Si ripete finché non si raggiunge l'unanimità.
- Carte speciali:
 - ? = non so stimare.
 - **Tazza di caffè** = pausa caffè
- **Team Estimation Game** (risolve il problema dell'àncora):
 1. **Fase 1**: ordinamento qualitativo (più facile/difficile), uno sviluppatore alla volta posiziona a destra o a sinistra la carta rispetto alle altre per dire se per lui ci vuole meno o di più.

2. **Fase 2:** assegnazione valori numerici (con carte del planning poker), serve a comprimere la riga della fase 1 assegnando un numero e raggruppando user stories con difficoltà simile.
3. **Fase 3:** eventuale stima in ore/uomo (spesso inutile).

3.6.2 Velocity

- Definizione: **story points completati per iterazione**.
- Non confrontabile tra team diversi (serve principalmente a dire al team quante task accorparsi in base al rendimento dell'iterazione precedente).
- Non deve includere storie incomplete.
- Può variare con l'aggiunta o rimozione di membri (l'aggiunta di un membro non la deve influenzare inizialmente poichè bisogna prima formarlo).
- Movimento **no estimates**: niente stime, solo esperienza e uniformità nelle storie (variante che funziona solo in team molto maturi).

3.7 Brevi cicli di rilascio

- Rilasci frequenti (ogni ~2 settimane).
- Ogni release deve essere **funzionante**.
- Cliente deve avere tempo di valutare e modificare richieste alla fine di ogni iterazione.
- Bertrand Meyer dice: *“Idea brillante, forse la più influente nell'industria agile”*.

3.8 Uso di una metafora

- Definisce un **vocabolario comune** cliente-sviluppatori.
- Sostituisce la documentazione come vista d'insieme.
- Facilita comprensione del sistema e scoperta di nuove feature.
- Difficile da creare nella pratica.

3.9 Semplicità di progetto

Arte di **massimizzare il lavoro non fatto**.

- Slogan XP:
 - **KISS**: Keep It Simple, Stupid.
 - **Once and one only**: nessuna duplicazione.
- Contrapposta al *design for change*, considerato un appesantimento inutile.
- Meglio chiedere al cliente cosa serve piuttosto che prevedere.
- La semplicità non implica necessariamente minore efficienza.

3.10 Testing

- Due tipi:
 - **Test di accettazione/funzionali**: scritti dai clienti nelle *user stories*.
 - **Test di unità**: scritti dai programmatori prima del codice.
- Testing esasperato: rete di protezione in tutte le fasi.
- Ogni cambiamento è verificabile tramite i test.
- Fondamentale anche nel refactoring e nella manutenzione di codice esistente.
- I test guidano lo sviluppo (**TDD**, Test-Driven Development) prima si scrive il test poi si scrive codice, si ha un'idea moltom più chiara di ciò che si deve fare.
- I test devono coprire tutte le righe di codice possibili.

3.11 Refactoring

- Modifica le **proprietà interne** del software, non le funzionalità.
- Obiettivo: ridurre l'entropia, aumentare leggibilità e semplicità.
- Deve essere continuo e graduale.

- Le modifiche devono essere sempre stimulate da test.
- Essenziale per garantire evolvibilità e qualità del codice.
- Coraggio necessario anche per i novizi nel modificare codice.
- Ristrutturazioni di sistemi legacy (modifiche molto grandi) possono richiedere strategie diverse.

3.12 Pair Programming (programmazione a coppie)

- Due sviluppatori per una singola postazione: **pilota** e **navigatore**.
- Vantaggi:
 - Controllo reciproco su codice e aderenza alle regole XP.
 - Migliore refactoring e condivisione continua delle idee.
 - Facilita inserimento di nuovi membri (apprendimento osmotico).
 - Aumenta la **proprietà collettiva** del codice.
 - I commenti “naive” aiutano a chiarire concetti dati per scontati.
- Produttività aumenta ~50% (inteso: se una persona ha produttività 100, due ne hanno insieme 150), ma la qualità complessiva migliora notevolmente.
- Riduce i tempi di verifica e validazione.

Critiche di Bertrand Meyer

- Pair programming utile, ma non deve essere imposto come metodo unico.
- Mancano evidenze empiriche conclusive sui vantaggi universali, quindi è più un’ideologia.
- Le persone sono diverse: alcuni programmatori lavorano meglio in solitudine.
- Forzare un solo modello di lavoro è rischioso in un’attività creativa.

3.13 Proprietà collettiva del codice

- Il codice appartiene all'intero team, non ai singoli.
- Nessuna policy di “code ownership”.
- Tutti possono modificare qualsiasi parte del codice.
- Le coppie cambiano più volte al giorno.
- Promuove conoscenza distribuita e collaborazione.

3.14 Integrazione continua

- Integrare frequentemente (più volte al giorno).
- L'integrazione deve essere graduale e sempre **consegna pronta**.
- Ogni feature completata deve essere integrata, testata e mostrata al cliente.
- Evita conflitti tra coppie e accumulo di problemi.
- Ogni coppia integra su una macchina comune con accesso esclusivo (token o simili).
- Se una coppia non riesce a integrare, deve ritirarsi e risolvere localmente.
- Tutte le coppie dispongono di una macchina personale di test.

3.15 Settimana di 40 ore

- Evitare straordinari e stress prolungato.
- Obiettivi:
 - Maggiore produttività e soddisfazione.
 - Migliore equilibrio vita-lavoro.
 - Riduzione turnover e burnout.
- Il lavoro intellettuale non è meccanico: si tende comunque a pensare ai problemi fuori orario.

3.16 Cliente sul posto

- Cliente presente fisicamente nel luogo di sviluppo.
- Vantaggi:
 - Maggior controllo per il cliente.
 - Possibilità di chiarire subito le specifiche.
 - Cliente come **documentazione vivente**.
- Deve essere rappresentativo di tutti gli stakeholder.
- Se non disponibile: introdurre un **Product Owner**, (un componente interno al team che si comporta come se fosse il cliente) (come in Scrum).
- Cliente può creare nuove *user stories* per iterazioni successive.

3.17 Standard di codifica

- Definire convenzioni comuni di scrittura del codice.
- Aumenta leggibilità e comunicazione tramite codice.
- Favorisce:
 - Refactoring.
 - Programmazione a coppie.
 - Proprietà collettiva.
- Strumenti automatici possono verificare o correggere lo stile.

3.18 They're just rules

- Regola aggiunta successivamente da alcuni agilisti.
- Al termine di ogni iterazione, il team può decidere:
 - Come comportarsi nella successiva.
 - Quali regole sospendere o riattivare.
- Decisione sempre a livello di **team**, mai del singolo.
- XP non è rigido: adattabile alle esigenze del gruppo.

- Tuttavia, le pratiche sono fortemente interconnesse e non vanno considerate isolate.

Raggruppamento in fasi

Requirements

- Clienti fanno parte del team: *requirements viventi*.
- Consegne incrementali e pianificazioni continue: evoluzione del progetto.

Design

- Metafora come visione unificante del progetto.
- Refactoring: design puro, consente evolvibilità del software.
- Presumere la semplicità.

Code

- Programmazione a coppie.
- Proprietà collettiva del codice.
- Integrazione continua.
- Standard di codifica.

Test

- Test di unità continuo, scritti prima del codice.
- Test funzionali scritti dagli utenti nelle *user stories*.

Spike

- Prototipi semplici per esplorare soluzioni.
- Aiutano a comprendere specifiche, tecnologia e interazione con componenti esterni.
- Creati, mostrati al cliente e infine scartati.

Documentazione

- La documentazione cartacea non è necessaria (cliente in loco, stories, compagno di peer prog.).
- Documentazione sostituita da:
 - Test di unità (specifiche eseguibili, scritte prima del codice).
 - Refactoring continuo per avere codice leggibile, eliminando commenti.

CRC Cards

- Class Responsibility and Collaboration cards rappresentano classi e relazioni.
- Uso simile al *planning game*, sono dall'occhio del programmatore.
- Contenuto di ciascuna carta:
 - Nome della classe
 - Superclassi e sottoclassi (se applicabile)
 - Responsabilità
 - Classi con cui collabora
 - Autore
- Piccole dimensioni per limitare complessità e responsabilità e consentire attività gestuali (spostare, riorganizzare, eliminare carte).
- Posizionamento delle carte può indicare relazioni di collaborazione o supervisione.

Criticità: Quando non utilizzare XP

- Agile non adatto se:
 - Team dislocati.
 - Team troppo numerosi.
 - Barriere tecnologiche (es. impossibilità integrazione continua).
 - Troppi stakeholder contrastanti.
 - Consegna incrementale non ha senso (es. centrali nucleari).

Critiche da Meyer

- Sottovalutazione del *up-front* design (a meno che non si è esperti, progettare dal basso è molto complesso).
- Sopravalutazione delle *user stories*.
- Mancata evidenziazione delle dipendenze tra *user stories* (dovrebbero essere indipendenti ma spesso è impossibile).
- TDD può portare a visione ristretta.
- Problemi nei team *cross-functional* troppo disomogenei (tante singole figure specializzate in un campo e queste devono collaborare in coppia).

Mesi uomo

stima spesso sbagliata per calcolare il tempo di sviluppo

- La stima in mesi-uomo non considera overhead di comunicazione.
- Aumentare il numero di persone non aumenta linearmente la produttività.
- In lavoro sequenziale o non parallelizzabile, più persone possono rallentare.
- Soluzioni in caso di ritardi:
 - Discutere la *dead-line* con il cliente.
 - Diminuire la portata del progetto.
 - Diminuire la qualità (*testing* ridotto, sconsigliato).
- Ideale: team di 8-10 persone; oltre questo, metodologie agili possono non funzionare.

Open Source

Definizione

- Software il cui **codice sorgente** è liberamente consultabile online.
- Sviluppo basato su una **community di sviluppatori indipendenti**, non su un team centralizzato.

Letteratura di riferimento

- *The Cathedral and the Bazaar* – Eric S. Raymond
- *Care and Feeding of FOSS* – Craig James
- *The Emerging Economic Paradigm of Open Source* – Bruce Perens
- *An empirical study of open-source and closed-source software products* – J.W. Paulson

The Cathedral and the Bazaar (Raymond)

Due metafore:

- **Cattedrale (closed source)**: struttura gerarchica, controllo centrale, un unico architetto.
- **Bazar (open source)**: ambiente anarchico, cooperazione spontanea, espansione continua.

Nascita di un progetto open-source

1. Uno sviluppatore ha un problema e crea un'applicazione per risolverlo.
2. Coinvolge amici/colleghi con lo stesso problema.
3. Si forma un piccolo gruppo informale che inizia lo sviluppo e si scambia conoscenze.
4. Si ottiene una prima versione funzionante.
5. Il progetto diventa **open-source** quando il codice è reso pubblico.
6. La community fornisce feedback, patch e nuove competenze.
7. Si entra in un ciclo di miglioramento continuo.

Concetti chiave

- “*Ogni buon lavoro software nasce dalla frenesia personale di un singolo sviluppatore.*”
- “*Se dai a tutti il codice sorgente, ognuno di essi diventa un tuo ingegnere.*”
- “*Se ci sono abbastanza occhi, gli errori diventano di poco conto.*”
- “*Tratta i beta-tester come la tua risorsa più importante.*”
- “*Quando perdi interesse in un programma, il tuo ultimo dovere è passarlo a un successore competente.*” (spesso non fatto e il progetto muore)

Confronto tra modelli di sviluppo

Aspetto	Tradizionale	Agile	Open Source
Documentazione	Strumento di controllo qualità	De-enfatizzata	Tutti i manufatti pubblici
Requisiti	Analisti traducono i bisogni	Utenti nel team	Sviluppatori = utenti
Assegnamento staff	1 progetto fisso	1 progetto fisso	Multi-progetto, vari livelli
Revisione codice	Rara	Pair programming	Necessaria, universale
Tempi di rilascio	Poche release grandi	Piccole release	Nightly, dev, stable
Organizzazione	Gestione dall'alto	Auto-organizzati	Ogni contributore decide
Testing	QA separato	TDD integrato	Tutti testano
Distribuzione lavoro	Parti assegnate	Tutti modificano tutto	Tutti modificano, commit limitati

Care and Feeding of FOSS (Craig James)

Ciclo di vita del software FOSS (Free and Open Source Software)

1. **Invenzione:** idea iniziale, esplosione di interesse e nuovi progetti.
2. **Espansione e innovazione:** competizione aziendale, troppe feature, non adatto per open source.
3. **Consolidamento:** il mercato si restringe, alcune aziende dominano.
4. **Maturità:** specifiche chiare, ottimo momento per nascita di progetti open (es. Linux).
5. **Transizione verso FOSS Domination:** le aziende iniziano a usare o integrare FOSS.
6. **FOSS Domination:** il prodotto open inizia a erodere i competitor chiusi.
7. **The FOSS Era:** dominio completo del mercato open (utopico, mai pienamente realizzato).

Esempi

- **Linux:** nato da Linus Torvalds dopo Minix, sfruttando specifiche mature.
- **IBM:** adotta Linux abbandonando il proprio OS.
- **Solaris:** reso open-source ma rapidamente dimenticato.

Conclusione

- L'open source eccelle in **continua innovazione e adattamento**.
- Le grandi aziende sono rallentate dal loro modello economico tradizionale.

The Emerging Economic Paradigm of Open Source (Bruce Perens)

Perché le aziende adottano l'open source

- Spesso il software è **una tecnologia abilitante**, non il prodotto.
- Sviluppo software = costo necessario, non fonte diretta di guadagno.

Vantaggio competitivo

- Un software proprietario ha senso solo se:
 1. Il cliente nota il suo effetto, se quella feature davvero ci distingue dagli altri e quindi è meglio tenerla privata.
 2. I competitor non hanno accesso allo stesso software.
- Se una delle due risposte è “no”, l'open source è preferibile:
 - Sparpaglia i costi.
 - Genera valore tramite contributi gratuiti.

Modelli di sviluppo software

- **Retail:** prodotto venduto a chiunque.
- **In-House & Contract:** software su commissione per un cliente specifico (azienda amazon).
- **Consortium & Non-Open Collaboration:** aziende concorrenti collaborano (raro).
- **Open Source:** codice pubblico e gratuito.

Criteri di confronto Efficienza, tasso di fallimento, costi di distribuzione, rischio di plagio, protezione del vantaggio competitivo, dimensione minima di mercato.

An empirical study of open-source and closed-source software products (Paulson)

Obiettivo Verificare empiricamente affermazioni comuni sul software open-source.

Risultati principali

- Nei progetti open-source si osserva una **maggiore frequenza di modifiche** alle funzioni.
- Non è possibile dedurre da ciò se:
 - gli errori siano trovati prima, (tante modifiche implicano che l'errore è stato visto prima di una situazione dove le modifiche sono più rare? non è detto);
 - i bug siano effettivamente risolti più rapidamente.
- Alcune credenze sull'open-source risultano **vere solo parzialmente**.

Sfide del modello open-source

1. Integrazione del software

- **Problema storico**, amplificato nel FOSS.
- Modelli precedenti:

- **A cascata**: integrazione fase separata e rigida.
- **Stabilize & Synchronize (Microsoft)**: compilazione notturna; se fallisce, si corregge; se riesce, nuova “versione migliore”.
- **XP**: integrazione continua, un solo sviluppatore alla volta.
- **Open Source**: integrazione anarchica e continua, senza coordinamento.
- **Rischio**: grandi cambiamenti improvvisi nella codebase.

2. Sfaldamento del team

- Difficoltà di comunicazione, coordinamento, coesione e reclutamento.
- Strumenti fondamentali:
 - Internet e forum per comunicare.
 - Wiki (in passato) o **Pages** per documentazione (Markdown, versioning con Git).
 - **Automazione delle build** (es. Makefile) per ambienti riproducibili.
 - **Bug tracking system** accessibili a tutti per evitare duplicati e gestire segnalazioni.
- Necessario educare gli utenti alla corretta segnalazione dei bug.

4 Software Configuration Management (SCM)

Le soluzioni di **Software Configuration Management** nascono per risolvere problemi molto comuni nello sviluppo software, come:

- Pubblicare un *hotfix* (correzione urgente di un bug per esempio) su una versione precedente a quella in sviluppo;
- Gestire accessi concorrenti e conflitti tra sviluppatori;
- Stabilire la responsabilità di ciascuna linea di codice.

L'SCM è l'insieme di pratiche che rendono sistematico il processo di sviluppo, tracciando ogni cambiamento in modo che il prodotto sia sempre in uno stato ben definito (**configurazione**) e che sia possibile recuperare facilmente versioni precedenti.

4.1 Storia

- **Anni '50:** nasce nel settore aerospaziale.
- **Anni '70–'80:** applicazione all'ingegneria del software.
- **Obiettivo:** mantenere revisioni controllate degli *artifact* e generare un prodotto da configurazioni definite.

4.2 Manufatti e configurazioni

Gli oggetti di cui si controlla l'evoluzione sono detti **configuration item** o **manufatti** (generalmente file). Cambiare nome a un file equivale ad eliminarlo e crearne uno nuovo.

Evoluzione degli strumenti

- **Anni '80:** strumenti locali (SCCS, RCS);
- **Anni '90:** strumenti client-server centralizzati (CVS, Subversion);
- **Anni '00:** strumenti distribuiti peer-to-peer (Git, Mercurial, Bazaar).

4.3 Centralizzato vs decentralizzato

centralizzato => lavorare online su un server (uno alla volta, bottleneck della rete)

decentralizzato => ognuno si può scaricare offline la repo

Il mondo **open source** preferisce approcci decentralizzati perché:

- Consentono di lavorare offline;
- Sono più veloci (nessun collo di bottiglia di rete);
- Supportano diversi modelli di lavoro:
 - Simil-centralizzato: un repository “di riferimento”;
 - Peer-to-peer: collaborazione diretta;
 - Gerarchico a più livelli (es. kernel Linux).

Non vi è sincronizzazione automatica: il merge tra repository è sempre esplicito. In Git è possibile anche usare algoritmi di merge personalizzati.

4.4 Meccanismo di base

Gli SCM non dipendono dal linguaggio di programmazione: lavorano a livello di file, trattando ogni linea come testo (eccezione: **Monticello** per Smalltalk(ambiente di sviluppo)).

Operazioni principali

- **Check-out**: dichiara l'intenzione di lavorare su una revisione specifica (come in biblioteca che ti registri per uscire con il libro);
- **Check-in** / **Commit**: registra una nuova revisione (restituzione del libro).

Il tutto avviene tramite lo scambio di dati tra:

- **Repository**: contiene versioni, etichette, branch, ecc.;
- **Workspace**: ambiente di lavoro locale.

4.5 Repository

La repository conserva differenze tra versioni (*delta*), ma in Git si usano **link simbolici** per velocità. Ogni versione è accessibile in tempo istantaneo.

Repository centralizzata vs distribuita

- **Centralizzata:** un server principale.
- **Distribuita:** ogni sviluppatore ha una copia completa, identificata tramite hash.

4.6 Cosa tracciare

Si tracciano:

- Codice sorgente;
- File di configurazione.

Non sempre conviene tracciare:

- Compilatori, librerie esterne (hanno già il loro sistema di versioning);
- File binari generati (molto grandi e ricalcolati dal sorgente dal compilatore).

Questo riduce la perfetta replicabilità ma aumenta l'efficienza. A volte i file generati vengono pubblicati come **package** (es. su GitHub/GitLab).

4.7 Accesso concorrente

Due modelli principali:

- **Pessimistico** (es. RCS): un solo utente può scrivere (lock);
- **Ottimistico** (es. CVS): si lavora in parallelo e si risolvono i conflitti con merge.

In Git, l'uso dei **branch** è incoraggiato e reso efficiente (es. politiche GitFlow).

4.8 SCM (Software Configuration Management) distribuito – vantaggi e limiti

Vantaggi

- Lavoro offline;
- Maggiore velocità;
- Supporto a modelli di collaborazione flessibili (peer, centralizzato, gerarchico).

Limiti

- Ogni peer ha il proprio repository;
- Nessuna sincronizzazione automatica → merge manuali richiesti.

4.9 Git

Struttura a 4 livelli

1. **Working Directory (WD)** — cartella di lavoro sul filesystem (file tracked e untracked);
2. **Index / Staging Area** — area temporanea dei file tracciati;
3. **Local Repository** — insieme dei commit e relativo storico;
4. **Remote Repository** — repository remota, può avere più branch o progetti.

Il passaggio tra i livelli è sempre esplicito (mai automatico).

Commit pointers

- Ogni **branch** punta al suo ultimo commit;
- **HEAD** punta al commit corrente;
- **HEAD scollegato**: quando si visita un commit precedente senza creare un branch.

4.9.1 `git commit` - record changes to the repository

- Salva il contenuto dall'index nel branch locale;
- Dopo il commit, **HEAD** e il puntatore del branch si aggiornano al nuovo commit;
- L'opzione `-amend` consente di sostituire l'ultimo commit.

4.9.2 `git switch` - switch branches

- Variante semplificata di `git checkout`;
- Cambia branch aggiornando **HEAD**, **index** e file della WD.

4.9.3 git merge - unire rami di sviluppo

- Unisce due (o più) branch:
 - Se non divergenti → **fast-forward merge**;
 - Altrimenti → **merge commit** o conflitto.
- Conflitti segnalati esplicitamente:

```
<<<<<< yours:sample.txt
Conflict resolution is hard;
let's go shopping.
=====
git makes conflict resolution easy.
>>>>>> theirs:sample.txt
```

- Dopo la risoluzione → commit conclusivo del merge.

3-way merge Git utilizza il **3-way merge**:

- Considera i due commit HEAD e il primo commit comune;
- Confronta le differenze nei file;
- Se la stessa parte è modificata in entrambi → conflitto manuale.

4.9.4 git reset - reset current HEAD

- Riporta HEAD (e opzionalmente la WD) ad un commit precedente;
- L'opzione **-soft** (Index e WD rimangono invariati)
- L'opzione **-mixed** (opzione di default, WD rimane invariata)
- L'opzione **-hard** ripristina anche i file locali.

—

→ Consultare la ***Git Cheatsheet*** per un riepilogo operativo dei comandi principali.

5 Git Workflow e Strumenti

5.1 Branch in Git

- Permettono versioni separate del codice.
- Libertà completa su creazione e nomi.
- Necessario stabilire politiche per team grandi.

5.2 GitFlow

- Organizzazione branch e merge guidato.
- Comandi shell integrati in script percepiti da Git.
- Branch principali:
 - **master**: versioni stabili pronte per produzione (ogni commit è una nuova versione, es: fine iterazione).
 - **develop**: integrazione delle feature (ogni commit è una nuova feature).
- Branch temporanei:
 - **feature**: nuove funzionalità o bug fix; partono da develop (un branch per feature).
 - **release**: preparazione rilascio, bug fix mirati; partono da develop (un branch serve per correggere bug generali per arrivare a una versione stabile per il master).
 - **hotfix**: correzioni urgenti; partono da master (ad esempio bug fix dopo il rilascio).

5.2.1 Feature Branch

- Isolano lavoro su funzionalità o bug.
- Merge in develop con: `git merge -no-ff feature/myFeature` (unisce al branch corrente il branch feature/myFeature)
- `-no-ff` evita fast-forward per tracciare inizio/fine feature (se non lo mettiamo e non ci sono conflitti, anche se aggiungiamo nuovi file, il risultato sarebbe solo di spostare il puntatore senza però creare un vero e proprio commit, per questo generalmente si mette).

- Squash opzionale (fusione di tutti i commit del branch feature uniti in un unico commit finale), ma consigliato *no* per preservare commit.

5.2.2 Release Branch

- Cristallizza funzionalità presenti in develop.
- Merge in master (rilascio) e develop (aggiornamento).
- Bug fix possibili con cherry-pick (prelevo un singolo commit da develop e non tutte le modifiche, ad esempio in un commit correggo un bug ma nei commit precedenti avevo introdotto molte funzionalità che potrebbero non essere ancora testate).
- Tagging con `git tag -a vX.Y.Z` (per segnare le versioni).

5.2.3 Hotfix Branch

- Correzioni urgenti su master e develop.
- Merge in master e develop, tag e branch eliminati alla fine.

5.3 Limiti di Git e GitFlow

- Mancanza di autorizzazioni granulari.
- Assenza di code review.
- Nessun sistema di comunicazione integrato.

5.4 git request-pull

- Genera una proposta di modifiche tramite email.
- Sostituito oggi dalle pull request web.
- Sintassi: `git request-pull [-p] <start> <URL> [<end>]`

5.5 Hosting Centralizzato

- Repository centrale per integrazione e controllo.
- GitHub/GitLab offrono fork, pull request, CI/CD e permessi.
- **Fork**: copia repository originale, indipendente ma collegata al padre.

5.6 Pull Request / Review

- Facilitano interazione e revisione.
- Possono generare conflitti se il codice evolve in parallelo.
- CI (Continuous integration) automatizza test poichè verifica automaticamente che il nuovo codice non rompa le funzioni già implementate segnalando subito possibili errori (i test non sono generati automaticamente), ma revisione manuale consigliata.

5.7 Gerrit

- Sistema di review peer-to-peer (Google, Android).
- Modifica accettata solo come unico commit (changeset).
- Figure chiave:
 - **Verifier**: verifica automatica patch.
 - **Approver**: approva validità, architettura, sicurezza, best practices.
- Architettura:
 - Authoritative repository: chiunque può solo leggere (quindi copiare la repo), solo dopo che i reviewer hanno approvato le modifiche (almeno 3 reviewer) le modifiche vengono aggiunte automaticamente nella repo attraverso la pending changes.
 - Pending Changes: tutti i developer possono fare la pull request, i reviewer revisionano e accettano o declinano.

5.8 Strumenti Open Source

- **Build automation**: make, Ant, Gradle.
- **Bug tracking**: git-bug, BugZilla, Scarab, GNATS, BugManager, Mantis.

5.8.1 Build Automation

- Automatizza compilazione e deploy.
- **make**: script shell, compilazione incrementale, gestione dipendenze.
- **Makefile**: definisce targets, dipendenze e comandi.
- Generazione automatica: automake, autoconf, imake.
- **Ant**: XML, Java, portabile su tutte le piattaforme (limite di makeFile dato che dipende da comandi di so, costruito per linux), deployment integrato, tasks con dipendenze.
- **Gradle**: Groovy/Kotlin, più di alto livello rispetto ad ant, multi-progetto, gestione dipendenze Maven, plugin per task.

5.8.2 Bug Tracking

- Serve ai programmatori per sapere come gestire un bug e segue un modus operandi preciso, infatti si apre una issue e si spiega esattamente cosa il bug provoca.
- Un bug poi verrà esaminato e potrà essere chiuso per i seguenti motivi:
 1. duplicate: errore già segnalato
 2. wontfix: errore che non si ha intenzione di risolvere(caratteristiche progettuali volute)
 3. can't reproduce: bug spiegato male o non riproducibile
 4. fixed: fixato ma non ancora inserito
 5. fix verified: fixato, testato e inserito
- bug considerato eliminato solo quando il fix entra a fare parte del master

5.9 Unified Process (UP / RUP)

- Meta-modello OO (Object Oriented) per sviluppo software.
- Sequenziale a 4 fasi:
 1. Inception: fase di studio, stima e stabilizzazione obiettivi
 2. Elaboration: stabilire architettura di base
 3. Construction: sviluppare, testare e integrare

4. Transition: fase di distribuzione al cliente e correzione di bug residui

- Iterativo: non si sviluppa tutto in una volta ma poco a poco.
- Incrementale: rilascio parziale e nuova iterazione.
- Workflow sovrapposti (4 fasi separate dove vengono eseguite le classiche fasi come: requirements, design, implementation.. tutte insieme ma in misure diverse, magari nella prima fase si fa molto requirements e design), milestone (concetto di raggiungimento della fine di ogni fase) per fine fase.
- Configurabile e adattabile, può diventare agile.

5.10 Refactoring e principi di design

Regole Refactoring:

- Non modificare funzionalità $\Rightarrow$ invisibile al cliente.
- Non inserire nuovi test dopo la fase verde.
- Se dura troppo $\Rightarrow$ rollback all'ultima versione verde e pianificare step multipli.
- *Regola "do it twice"*: il secondo tentativo è più veloce e migliore.

Perchè dovrei farlo?:

- Design complesso/poco leggibile.
- Preparare il design per nuove feature difficili da integrare (*design for change*).
- Presenza di *debito tecnico*: scorciatoie che rallentano l'evoluzione.

Design Knowledge (come tenere traccia di come è strutturato il codice):

- **Memoria**: non affidabile nel tempo.
- **Documenti di design (diagrammi)**: si desincronizzano se non aggiornati.
- **Piattaforme di discussione**: info sparse, difficile aggiornamento.
- **UML e generative programming**: inefficaci nella pratica.
- **Codice**: fonte più affidabile, ma difficile capirne le motivazioni.

SOLUZIONE: Combinare più tecniche; documentare le ragioni delle scelte, non le scelte.

Condivisione del design:

- **Metodi**: (es. Agile, OO).
- **Design pattern**: linguaggio comune, riusabilità, soluzioni note.
- **Principi**: es. SOLID.

6 Conoscenze Preliminari

6.0.1 Linguaggio Object Orientation

Tre proprietà fondamentali:

- **Ereditarietà:** eredita proprietà e comportamenti.
- **Polimorfismo:** classi assumono forme diverse in base alle interfacce.
- **Collegamento dinamico:** in Java avviene a runtime; in C++ serve `virtual` (operazione per realizzare il polimorfismo).

6.0.2 Principi SOLID

- **S - Single Responsibility:** una classe, un solo scopo.
- **O - Open/Closed:** estendibile senza modifiche interne.
- **L - Liskov Substitution:** figlie rispettano contratti della classe padre.
- **I - Interface Segregation:** tante interfacce piccole, non una grande.
- **D - Dependency Inversion:** dipendere da astrazioni, non da concreti.

6.0.3 Reference Escaping (errore da non fare)

Violazione dell'incapsulamento:

- Getter che restituisce riferimento privato.
- Setter/costruttore che assegnano riferimenti esterni (se si memorizza il riferimento all'oggetto passato, posso modificare quell'oggetto esternamente, se invece ne faccio una copia non posso).

```
1      public void setDataInizio(Date nuovaData) {  
2          if (nuovaData == null) {  
3              throw new IllegalArgumentException("  
4                  La data non puo essere null");  
5          }  
6          // Copia difensiva per prevenire il  
7          // reference escaping  
8          this.dataInizio = new Date(nuovaData.  
9              getTime());  
10     }
```


6.0.4 Encapsulation e Information Hiding

Legge di Parnas: solo ciò che è nascosto può essere modificato liberamente. Separare chiaramente cosa è pubblico e cosa è privato.

6.0.5 Legacy e Deprecated

- **Legacy:** supportata ma sostituita da una più recente.
- **Deprecated:** funzionante ma non supportata; da sostituire con nuove API.

6.0.6 Immutabilità

Classe immutabile $\Rightarrow$ stato non modificabile dopo la creazione.

- Niente metodi mutatori.
- Attributi privati e/o final.
- Getter che non restituiscono riferimenti.
- Nessun reference escaping.

6.0.7 Code Smell

Segnali di cattivo design (debito tecnico):

- Codice duplicato.
- Metodi lunghi / troppi livelli di indentazione.
- Troppi attributi.
- Sequenze di if/switch $\Rightarrow$ preferire polimorfismo.
- Liste di parametri lunghe.
- Numeri magici $\Rightarrow$ usare costanti nominate.
- Commenti esplicativi $\Rightarrow$ codice poco chiaro.
- Nomi oscuri o incoerenti.
- Codice morto o non testato.
- Getter/Setter $\Rightarrow$ perdita di incapsulamento.

6.0.8 Tell-Don't-Ask

Non chiedere i dati, ma dì cosa fare con essi. L'oggetto deve gestire le proprie informazioni ed esporre solo operazioni, non dati grezzi.

6.0.9 Interface Segregation (approcci pratici)

- **Up front:** scrivere direttamente l'interfaccia.
- **Down front:** scrivere il codice e poi estrarne l'interfaccia (più adatto al TDD).

6.1 Esempio: gerarchia Card / Deck (up front)

Concetti trattati: Interface segregation, collegamento statico/dinamico, loose coupling, interfacce multiple, contract-based design vs programmazione difensiva, classi astratte.

Interface Segregation all'opera

- Metodo:

```
1      public static List<Card> drawCards(CardSource
2          deck, int n) {
3          List<Card> result = new ArrayList<>();
4          for (int i = 0; i < n && !deck.isEmpty(); i++)
5              result.add(deck.draw());
6          return result;
7      }
```

- Deck implementa solo le capacità necessarie: `isEmpty()` e `draw()`.
- L'interfaccia `CardSource` astrae tali capacità, permettendo riuso generico.

Collegamento statico e dinamico

- In Java $\Rightarrow$ tipo dichiarato staticamente: `Deck implements CardSource`.
- In Go $\Rightarrow$ più dinamico: basta avere metodi compatibili (non serve dire che un oggetto è sottotipo di un altro, basta che il metodo abbia lo stesso nome).

- Rischio duck typing: nomi poco specifici $\Rightarrow$ ambiguità (es. `play()` in contesti diversi).

Loose Coupling

- Permette di assegnare a delle variabili generali dei tipi diversi a patto che siano dei loro sottotipi:

```
1 Deck deck = new Deck();  
2 CardSource source = deck;  
3 List<Card> cards = drawCards(deck, 5);
```

- Favorisce flessibilità ed estendibilità.

Interfacce Multiple

- Deck può implementare più interfacce (`CardSource`, `Shuffable`, `Iterable<Card>`).
- Ogni metodo utilizza solo le capacità realmente necessarie.

Contract-based Design vs Programmazione Difensiva

- **Contract-based:** contratto tra chiamante e chiamato.
 - Il chiamante deve rispettare le precondizioni (`@pre !isEmpty()`).
 - Violazione del contratto = errore logico.
- **Asserzioni:** utili in sviluppo, rimosse in produzione.
- **Programmazione difensiva:** la verifica è a carico del chiamato, non del chiamante.

Classi Astratte

- Possono implementare parzialmente un'interfaccia e delegare il resto alle sottoclassi.
- Non istanziabili; migliorano la generalità ma con lieve perdita di performance.
- Utili anche per classi complete ma concettualmente non istanziabili (es. utility).

Esempio: Classi Utility Java

- `Collections.shuffle(List<?> list)`: ignora il tipo, serve solo mischiare.
- `Collections.sort(List<T> list)` richiede:

```
public static <T extends Comparable<? super T>> void sort(List<T> list)
```

- `Comparable<T>` specifica che un oggetto è comparabile.
- `Comparable` è un esempio di *interface segregation*: un'interfaccia per una singola capacità.

Digressione finale

- La classe `Collections` era un modo per aggiungere metodi alle interfacce.
- Oggi si possono usare i `default methods` direttamente nelle interfacce (generalmente le interfacce non contengono il corpo della funzione, `default` permette di farlo).

7 Design Patterns

7.1 Definizione generale

- Soluzioni concettuali e riutilizzabili a problemi di design ricorrenti.
- Non codice ma **architetture di progettazione** riconosciute efficaci.
- Ogni pattern deve avere un nome evocativo per favorire la memorizzazione.
- Gli **idiomi** sono implementazioni concrete del pattern in uno specifico linguaggio.
- Esistono anche gli **anti-patterns**, ovvero soluzioni apparentemente buone ma dimostrate errate.

7.2 Categorie dei pattern (GoF)

- **Creazionali**: gestione della creazione di oggetti.
- **Strutturali**: composizione di classi e oggetti.
- **Comportamentali**: interazione e comunicazione tra oggetti.

7.3 Meta-patterns

- Pattern per **costruire altri pattern**.
- Due elementi fondamentali:
 - **Hook Method**: metodo astratto che rappresenta il punto “caldo” modificabile.
 - **Template Method**: struttura generale che usa gli hook per definire la parte variabile.

Un esempio potrebbe essere una funzione di ordinamento, dove c'è uno scheletro generale dell'algoritmo (template) al cui interno non viene specificato il modo in cui avviene la comparazione tra elementi. Customizzando questa comparazione si realizzano gli hook, che se inseriti all'interno del template rendono l'algoritmo funzionante.

- Collegamento Hook–Template:
 - **Unification:** hook e template nella stessa classe (es. `Iterable` in Java che richiede l’implementazione dei metodi `next` e `hasNext`, inoltre è presente anche un metodo di default chiamato `forEach` che permette di chiamare `next` fintantochè `hasNext` restituisce `true`. Tutti questi metodi si trovano nella stessa interfaccia).
 - **Connection:** hook e template in classi diverse collegate per aggregazione (classe template che contiene la classe hook).
 - **Recursive Connection:** hook e template in classi collegate da generalizzazione (es. `Decorator`, `Composite`).

7.4 Singleton

- Garantisce l’esistenza di una sola istanza di una classe.
- Implementazione classica:
 - Costruttore privato/protetto.
 - Metodo statico `getInstance()` che crea e restituisce l’unica istanza (ontrollando che non sia già stata istanziata).

```

1 public class Singleton {
2     /* costruttore privato o comunque non pubblico */
3     protected Singleton() { ... }
4
5     /* salvo l'istanza per usarla dopo */
6     private static Singleton instance = null;
7
8     /* metodo statico */
9     public static Singleton getInstance() {
10         if (instance == null) {
11             instance = new Singleton();
12         }
13         return instance;
14     }
15
16     public void metodoIstanza() { ... }
17 }

```

- Problema: concorrenza (se nello stesso momento si accede al metodo possono esserci duplicati) → uso del **Double Check Pattern** con blocco `synchronized`, non si sincronizza tutto il metodo ma solo una parte dopo la verifica per vedere se è già stata istanziata o meno, il tutto per non far calare troppo le prestazioni.

```
1 public class Singleton {
2     private static Singleton instance = null;
3
4     protected Singleton() {}
5
6     public static Singleton getInstance() {
7         if (instance == null) {
8             synchronized(Singleton.class) {
9                 if (instance == null)
10                    instance = new Singleton();
11            }
12        }
13        return instance;
14    }
15    public void sampleOp() {...}
16 }
```

- **Idioma Java:** uso di un enum con un solo valore (INSTANCE), thread-safe.

```
1     public enum MySingleton {
2         INSTANCE;
3
4         public void metodoIstanza() { ... }
5     }
6
7     MySingleton.INSTANCE.sampleOp();
```

- Svantaggi: può violare principi SOLID e fungere da variabile globale nascosta.

7.5 Iterator

- Permette di scorrere elementi di una collezione senza esporne la rappresentazione interna.
- Interfacce:
 - `Iterator<E>`: definisce `hasNext()`, `next()`, `remove()`.
 - `Iterable<E>`: definisce `iterator()`, abilita il costrutto `for-each` (in pratica il compilatore traduce il `for each` con la sintassi dell'iteratore se abbiamo implementato `iterable`, questo però può essere fatto solo su un tipo).
- Problema: `remove()` viola SRP → sarebbe meglio separarlo.
- Una classe non può essere iterabile su più tipi diversi (type erasure).

7.6 Chain of Responsibility

- Gestisce richieste sequenziali in una **catena di gestori**.
- Ogni gestore decide se elaborare la richiesta o passarla al successivo.
- Evita lunghi blocchi di `if/else` e semplifica estendibilità.
- Implementazione:
 - Interfaccia `Gestore` con metodo `evaluate()`.
 - Ogni gestore contiene un riferimento al successivo (`next`).
 - L'ultimo può usare un `NullObject` come terminatore.

```
1 public interface Gestore {
2
3     /* Il tipo di ritorno dipende dal campo applicativo
4        */
5     public ??? evaluate();
6 }
7
8 public class Client {
9
10     private Gestore evaluator =
11     new GestoreConcreto1(
```



```

12     new GestoreConcreto2(
13     new GestoreConcreto3(null)));
14
15     public void richiesta() {
16         evaluator.evaluate();
17     }
18
19 }
20
21 public class GestoreConcreto {
22
23     private evaluator next;
24
25     public GestoreConcreto(evaluator next) {
26         this.next = next; //next potrebbe anche essere
           null
27     }
28
29     public GestoreConcreto() {
30         this.next = null;
31     }
32
33
34     public ??? evaluate() {
35         if thisIsTheCase() return "Questo evaluatore
           gestisce il caso";
36         else
37         if next != null return next.evaluate(); //provo
           con il successivo
38         else return "Non sono in grado di gestire questo
           caso";
39         //sono arrivato all'ultimo caso e non sono stato
           in grado di gestirlo
40     }
41
42 }

```

7.7 Flyweight

(simile a singleton, la diff è che flyweight ha uno stato interno immutabile condiviso con altre classi per risparmiare memoria)

- Gestisce oggetti immutabili condivisi per ridurre uso di memoria.
- Costruttore privato, creazione di tutte le istanze in anticipo, archiviate in struttura statica.
- I client richiedono istanze tramite un metodo `get(...)` che restituisce sempre la stessa.
- Esempio: classe `Card` con tutte le combinazioni di semi e valori.
- Vantaggio: efficienza in memoria; svantaggio: inizializzazione pesante.

```
1 public class Card {
2     private static final Card[][] CARDS = new Card[Suit.
        values().length][Rank.values().length];
3     // l blocco static { ... } esegue l'esatto compito di
        un costruttore, ma a livello di classe
4     static {
5         for (Suit suit : Suit.values()) {
6             for (Rank rank : Rank.values()) {
7                 CARDS[suit.ordinal()][rank.ordinal()] =
                        new Card(rank, suit);
8             }
9         }
10    }
11
12    public static Card get(Rank pRank, Suit pSuit) {
13        return CARDS[pSuit.ordinal()][pRank.ordinal()];
14    }
15 }
```

7.8 Null Object

- Evita l'uso esplicito di `null` e le `NullPointerException`.
- Approcci:
 - Controlli condizionali o asserzioni.
 - Annotazioni (`@NotNull`).

- Uso di **Optional**<T> per rappresentare valori assenti.

```
1 public interface CardSource {
2     Card draw();
3     boolean isEmpty();
4     // istanza che si auto definisce null con la
      logica dell'interfaccia
5     public static CardSource NULL = new CardSource()
6     {
7         public boolean isEmpty() {
8             return true;
9         }
10        public Card draw() {
11            assert !isEmpty();
12            return null;
13        }
14    }
```

- Pattern **NullObject**: definisce un'istanza statica NULL con comportamento neutro. (in pratica quando si implementa questa interfaccia, basterà scrivere cardSource.null per avere lo stesso oggetto ma nullo)
- Mantiene l'identità del tipo e impedisce eccezioni a runtime.

7.9 Strategy (Delegation)

- Definisce una **famiglia di algoritmi** intercambiabili a runtime.
- Sostituisce l'ereditarietà con la composizione.
- Ogni comportamento è incapsulato in un oggetto **Strategy** (se duck ha sia volare che verso, è possibile che alcune anatre non possano volare e altre sì, per evitare di implementare fly in ciascuna anatra, si separano inn due interfacce diverse i metodi comuni a tutti e quelli opzionali, stabilendo delle politiche di comportamento implementabili).
- La classe “client” delega l'azione a un oggetto strategia (es. FlyBehaviour nelle Duck).
- Vantaggi:
 - Maggiore flessibilità e mantenibilità.
 - Rispetto dei principi Open/Closed e Liskov.

7.10 Observer

- Mantiene sincronizzati più oggetti dipendenti da uno stato comune.
- Evita l'anti-pattern delle **pairwise dependencies** (3 classi che condividono uno stato (a dipende da b e b da c e c da a), se cambio lo stato di una devo cambiare lo stato anche delle altre due) questo perché lo stato è distribuito.
- Struttura:
 - **Subject** (osservato) mantiene lo stato e la lista di osservatori.
 - **Observer** si registra e viene notificato ai cambiamenti.
- Architettura **event-driven**: il Subject notifica, non viene interrogato.
- due modi di farlo:
 - **Push**: invio direttamente tutto lo stato senza passare l'istanza dell'observer (PROBLEMA: stato magari molto pesante, quindi inutile passarlo tutto)

```
1  // Observable
2  @Override
3  public void notifyObservers() {
4
5      for (Observer observer : observers) {
6          observer.update(null, state);
7      }
8
9  }
10
11 // Observer
12 @Override
13 public void update(Observable model, Object state
14 ) {
15
16     if (state instanceof Integer intValue) {
17         doSomethingOn(intValue);
18     }
19 }
```

- Pull: passo solo l'istanza dell'observable e nessun stato in modo da permettere all'observer di prendere solo ciò che gli interessa con get dei vari stati (PROBLEMA: serve fare casting per verificare che l'observable passato sia quello richiesto (model instanceof ConcreteObservable cModel))

```

1 // Observable
2 @Override
3 public void notifyObservers() {
4
5     for (Observer observer : observers) {
6         observer.update(this, null);
7     }
8
9 }
10
11 // Observer
12 @Override
13 public void update(Observable model, Object state
14 ) {
15
16     if (model instanceof ConcreteObservable
17         cModel) {
18         doSomethingOn(cModel.getState());
19     }
20 }

```

- approccio ibrido: spesso si usa un approccio ibrido tra pull e push permettendo di passare lo stato più rilevante e permettere l'estrazione dello stato che ci interessa
- risolvere casting: spesso si mette un attributo in observer che rappresenta l'observable per evitare il casting avendo solo il controllo:

```

1 public class ConcreteObserver {
2     private Observable mySubject;
3
4     @Override
5     public void update(Observable subject, Object
6         stato) {
7         if (subject == mySubject) {
8             ...
9         }
10    }
11 }

```

```

8         }
9     }
10 }

```

- In Java: interfacce `Observer`, `Observable` (deprecate, non thread-safe).

7.11 Adapter

Scopo: rendere compatibili interfacce o componenti non originariamente progettati per collaborare (es. librerie COTS o codice legacy). Permette l'integrazione incrementale e controllata di parti eterogenee.

Tipi:

- **Class Adapter:** adatta una *classe*.
- **Object Adapter:** adatta un *oggetto*.

7.11.1 Class Adapter

Idea: la classe `Adapter` **estende** `Adaptee` (vecchia classe da adattare) e **implementa** `Target` (interfaccia con i nuovi metodi che adatteranno la vecchia classe). Il client usa l'`Adapter` tramite l'interfaccia `Target`, ma le chiamate vengono inoltrate ai metodi della classe adattata.

```

1 public class Adapter extends Adaptee implements Target {
2     @Override
3     public void request() {
4         this.oldRequest();
5     }
6 }

```

Pro:

- Accesso diretto a metodi/attributi protetti dell'`Adaptee`.
- Riutilizzo del codice esistente.
- Possibilità di usare l'istanza con entrambe le interfacce.

Contro:

- Problemi con l'ereditarietà multipla (es. Java).

- Difficile adattare un'interfaccia (nessuna ereditarietà di codice dato che le interfacce non contengono implementazione).
- Possibili conflitti tra nomi di metodi identici ma con comportamento diverso.

7.11.2 Object Adapter

Idea: la classe **Adapter** **contiene** (composizione) un'istanza di **Adaptee** e **delega** le chiamate a essa.

```

1      public class Adapter implements Target {
2          private final Adaptee adaptee;
3          public Adapter(Adaptee adaptee) { this.
            adaptee = adaptee; }
4          @Override
5          public void request() { adaptee.oldRequest();
            }
6      }

```

Pro:

- Supporta adattamento di interfacce (non usa ereditarietà).
- Nessun problema di ereditarietà multipla.
- Può adattare intere gerarchie di classi.

Contro:

- Richiede due istanze (Adapter + Adaptee) → maggiore uso memoria.
- Nessun accesso ai membri protetti dell'Adaptee.
- Occorre reimplementare tutti i metodi (delega esplicita).

7.11.3 Confronto Class vs Object Adapter

Aspetto	Class Adapter	Object Adapter
Accesso all'Adaptee	Diretto (protetti e pubblici)	Solo tramite interfaccia pubblica
Riuso del codice	Elevato (no reimplementazione)	Basso (delega esplicita)
Uso della memoria	1 istanza	2 istanze
Uso interfacce	Vecchia e nuova	Solo nuova
Ereditarietà multipla	Problematica	Nessuna
Adattamento interfacce	Non indicato	Ottimo, anche per gerarchie

Facade

- **Problema:** un sistema complesso può esporre molte interfacce specifiche e dettagliate, costringendo il client a interagire con più sottosistemi anche per operazioni semplici.
- **Soluzione:** fornire una **interfaccia unificata e semplificata** (la Facade) che nasconde la complessità interna combinando varie funzionalità.
- **Scopo:** semplificare l'uso del sistema e rendere l'accesso alle operazioni più frequenti immediato, mantenendo comunque la possibilità di usare le interfacce più dettagliate.
- **Esempi:**
 - **Telecomando:** controlla TV con comandi semplici, ma consente accesso a funzioni avanzate.
 - **GitFlow:** comandi come `init`, `feature start/finish` o `release` eseguono più operazioni (branch, merge, ecc.) dietro un'interfaccia semplice.

Composite

- **Problema:** modellare strutture gerarchiche (es. file system con File e Directory) su cui le stesse operazioni (crea, cancella, calcola dimensione) devono funzionare in modo uniforme.
- **Soluzione:** introdurre il pattern **Composite**, che gestisce oggetti singoli e composti in modo uniforme tramite una **interfaccia comune** (Component).
- **Struttura:**
 - **Component:** interfaccia comune per operazioni.
 - **Leaf:** oggetto singolo.
 - **Composite:** contiene 0..n **Component** (Leaf o Composite), implementa operazioni comuni delegandole ai componenti interni.
- **Cardinalità:**
 - **Composite** → **Component**: 0..n (può contenere più componenti).

- Component → Composite: 0..1 (ogni componente può appartenere a un solo Composite).
- **Vantaggi:** gestione uniforme di oggetti e gruppi; codice più semplice per operazioni ricorsive.
- **Svantaggi:** impossibilità di distinguere chiaramente tra Leaf e Composite; nessun controllo sul contenuto o sulla profondità dell'albero.
- **Soluzioni per distinguere i tipi:**
 - Metodo che ritorna il tipo reale (`this` o `null`) → poco pulito.
 - Casting controllato con `instanceof` → accettabile ma non ideale.
 - Spostare tutte le operazioni nell'interfaccia e sollevare eccezioni nelle Leaf → poco elegante.
- **Esempio: Deck composito**
 - CardSourceComp contiene lista di CardSource.
 - `isEmpty()` restituisce vero solo se tutti i sotto-mazzi sono vuoti.
 - Il metodo `draw()` diventa più complesso (richiede scelta casuale di mazzo).

```

1  public interface CardSource {
2      Card draw();           // Per prelevare una carta
3      boolean isEmpty();    // Per sapere se e' vuoto
4  }

```

```

1  public class CardSourceComp implements CardSource {
2      @NotNull private List<CardSource> sources;
3
4      public CardSourceComp(@NotNull List<CardSource>
5          sources) {
6          this.sources = sources;
7      }
8      @Override
9      public boolean isEmpty() {
10         for(CardSource source : sources) {
11             if(!source.isEmpty()) return false;
12         }
13         return true;
14     }

```

```

14  @Override
15  public Card draw() {
16      for (CardSource source : sources) {
17          if (!source.isEmpty()) {
18              return source.draw();
19          }
20      }
21      return CardSource.NULL.draw();
22  }
23  }

```

```

1  public class SingleDeck implements CardSource {
2      private final String name;
3      private int cardCount;
4
5      public SingleDeck(String name, int initialCount)
6      {
7          this.name = name;
8          this.cardCount = initialCount;
9      }
10
11     @Override
12     public Card draw() {
13         if (cardCount > 0) {
14             cardCount--;
15             System.out.println(name + " ha pescato
16                 una carta. Rimanenti: " + cardCount);
17             return new Card();
18         }
19         return CardSource.NULL.draw();
20     }
21
22     @Override
23     public boolean isEmpty() {
24         return cardCount == 0;
25     }
26 }

```

Decorator

- **Problema:** aggiungere dinamicamente funzionalità a un oggetto senza esplosione di classi o violare l'Open-Closed Principle.
- **Soluzioni errate:**
 - Gerarchia statica di classi per ogni combinazione → **anti-pattern** (esplosione combinatoria).

```
1 public interface Pizza {}
2 public class BaseNormale implements Pizza {
3     public String toString() {
4         return "Sono una pizza con: base normale"
5     }
6 }
7 public class BaseIntegrale implements Pizza {
8     public String toString() {
9         return "Sono una pizza con: base
10             integrale";
11     }
12 }
13 public class BaseNormaleSalame extends
14     BaseNormale {
15     public String toString() {
16         return "Sono una pizza con: base normale,
17             salame";
18     }
19 }
20 public class BaseNormaleSalamePeperoni extends
21     BaseNormaleSalame {
22     public String toString() {
23         return "Sono una pizza con: base normale,
24             salame, peperoni";
25     }
26 }
```

- God Class con flag booleani → violazione OCP, scarsa manutenibilità.

```
1 public class GodPizza {
2     boolean baseNormale = false;
3     boolean baseIntegrale = false;
4     ...
}
```

```

5      boolean salame = false;
6      boolean pancetta = false;
7      boolean peperoni = false;
8      ...
9
10     public void setBaseNormale(boolean status) {
11         baseNormale = status; }
12     public void setBaseIntegrale(boolean status)
13         { baseIntegrale = status; }
14     ...
15
16     public void setSalame(boolean status) {
17         salame = status; }
18     public void setPancetta(boolean status) {
19         pancetta = status; }
20     public void setPeperoni(boolean status) {
21         peperoni = status; }
22     ...
23
24     public String toString() {
25         StringBuilder sb = new StringBuilder("
26             Sono una pizza con: ");
27         if (baseNormale) sb.append("base normale,
28             ");
29         if (baseIntegrale) sb.append("base
30             integrale, ");
31         ...
32         if (salame) sb.append("salame, ");
33         if (pancetta) sb.append("pancetta, ");
34         if (peperoni) sb.append("peperoni, ");
35         ...
36         sb.removeCharAt(sb.length() - 1);
37         sb.removeCharAt(sb.length() - 1);
38         return sb.toString();
39     }
40 }

```

- **Soluzione:** introdurre **Decorator Pattern**.
- **Struttura:**
 - Component: interfaccia comune.
 - ConcreteComponent: base (es. BaseNormale).

- Decorator (astratto): implementa Component, contiene riferimento a un altro Component.
- ConcreteDecorator: aggiunge funzionalità prima/dopo le chiamate al componente interno.

```

1 public interface Pizza { String toString(); } //
   component
2
3 public class BaseNormale implements Pizza { //
   concreteComponent
4     public String toString() {
5         return "Io sono una pizza con: base normale";
6     }
7 }
8
9 public class BaseIntegrale implements Pizza { //
   concreteComponent
10    public String toString() {
11        return "Io sono una pizza con: base integrale
12            ";
13    }
14 }
15 public abstract class IngredienteDecorator implements
   Pizza { //decorator
16     private Pizza base;
17
18     public IngredienteDecorator(Pizza base) { this.
        base = base; }
19
20     public String toString() {
21         return base.toString();
22     }
23 }
24
25 public class IngredienteSalame extends
   IngredienteDecorator { //concreteDecorator
26     public IngredienteSalame(Pizza base) { super(base
        ); }
27
28     @Override
29     public String toString() { return super.toString
        () + ", salame"; }

```

```

30 }
31
32 public class IngredientePeperoni extends
    IngredienteDecorator { //concreteDecorator
33     public IngredientePeperoni(Pizza base) { super(
        base); }
34
35     @Override
36     public String toString() { return super.toString
        () + ", peperoni"; }
37 }

```

e poi il client:

```

1 public class Client {
2     public static void Main() {
3         // Voglio una pizza con salame, peperoni e
4         base integrale
5         Pizza salamePeperoni =
6             new IngredientePeperoni(
7                 new IngredienteSalame(
8                     new BaseIntegrale()
9                 )
10            );
11 }

```

- **Differenze con Composite:**

- **Composite:** crea oggetti composti.
- **Decorator:** aggiunge funzionalità dinamicamente.

- **Caratteristiche:**

- Ogni Decorator può decorare altri Decorator → struttura ricorsiva.
- Escaping reference intenzionale (il Decorator deve riflettere cambiamenti dell'oggetto base).
- Facilità di estensione grazie a classe astratta **Decorator** con metodo **protected**.

- **Uso combinato con Composite:** possibile decorare gruppi di oggetti o gruppi decorati (es. effetti multipli in photo-editing).

- **Esempio reale:** InputStream di Java.

State (macchine a stati finiti)

- **Scopo:** modellare oggetti il cui comportamento varia in base allo stato corrente.
- **Problema:** evitare uso di lunghi if/switch per gestire stati.
- **Struttura:**
 - **State:** interfaccia con metodi che rappresentano azioni.
 - **ConcreteState:** implementa comportamento specifico per ogni stato.
 - **Context:** mantiene riferimento allo stato corrente e delega a esso le operazioni.
- **Transizioni:**
 - **Gestite dallo State:** più coerente con automi a stati finiti, ma introduce dipendenza tra stati.
 - **Gestite dal Context:** centralizza le transizioni, ma reintroduce if/switch.
- **Preferenza:** farle gestire dagli State → passaggi di stato atomici e chiari.
- **Proprietà:**
 - Gli State sono riutilizzabili tra Context diversi.
 - Ogni Context mantiene solo il riferimento allo stato corrente.
- **Differenze con Strategy:**
 - Gli State conoscono gli altri stati.
 - Le Strategy sono indipendenti.

```
1 // L'interfaccia State
2 public interface TrafficLightState {
3     void handle(TrafficLightContext context);
4 }
```

```

1 // ConcreteState
2 public class RedState implements TrafficLightState {
3     @Override
4     public void handle(TrafficLightContext context) {
5         System.out.println("Semaforo ROSSO: Fermati."
6             );
7         // Logica di transizione: il prossimo stato e
8         // Verde
9         context.setState(new GreenState());
10    }
11 }
12
13 // ConcreteState
14 public class GreenState implements TrafficLightState
15 {
16     @Override
17     public void handle(TrafficLightContext context) {
18         System.out.println("Semaforo VERDE: Vai.");
19         // Logica di transizione: il prossimo stato e
20         // Giallo
21         context.setState(new YellowState());
22    }
23 }
24
25 // ConcreteState
26 public class YellowState implements TrafficLightState
27 {
28     @Override
29     public void handle(TrafficLightContext context) {
30         System.out.println("Semaforo GIALLO:
31             Attenzione, preparati a fermarti.");
32         // Logica di transizione: il prossimo stato e
33         // Rosso
34         context.setState(new RedState());
35    }
36 }

```

```

1 // La classe Context (Il Semaforo)
2 public class TrafficLightContext {
3     private TrafficLightState state;
4     // Inizializzazione con lo stato iniziale (es. Rosso)
5     public TrafficLightContext() {

```



```

6         this.state = new RedState();
7     }
8
9     // Metodo per cambiare lo stato (usato dalle
10    // classi State)
11    public void setState(TrafficLightState newState)
12    {
13        this.state = newState;
14    }
15
16    // Il metodo di operazione delega al metodo dell'
17    // oggetto State attuale
18    public void change() {
19        // La chiamata a handle() attiva il
20        // comportamento specifico dello stato
21        state.handle(this);
22    }
23 }

```

Factory Method

- **Problema:** creare oggetti senza conoscere la loro classe concreta, ma solo l'interfaccia (devo solo conoscere la fabbrica del veicolo, così facendo se il veicolo cambia, il client non lo deve sapere).
- **Soluzione:** introdurre un **metodo fabbrica virtuale** che delega la creazione a sottoclassi.
- **Struttura:**
 - Creator (astratto): definisce `factoryMethod()` che restituisce un `Product`.
 - ConcreteCreator: implementa il metodo fabbrica, creando un `ConcreteProduct`.
 - Product: interfaccia comune.
 - ConcreteProduct: implementazione reale.

```

1 //abstract creator
2 public abstract class VeicoloFactory {
3     public abstract Veicolo creaVeicolo();
4
5     public String pianificaLogistica() {

```

```

6         Veicolo veicolo = creaVeicolo(); //
           Chiamata al factory Method
7         return "Logistica pianificata. Inizio: "
           + veicolo.guida();
8     }
9 }

```

```

1 // concrete creator
2 public class AutoFactory extends VeicoloFactory {
3     @Override
4     public Veicolo creaVeicolo() {
5         return new Auto(); // Istanzia il prodotto
           specifico: Auto
6     }
7 }
8 // concrete creator
9 public class CamionFactory extends VeicoloFactory {
10     @Override
11     public Veicolo creaVeicolo() {
12         return new Camion(); // Istanzia il prodotto
           specifico: Camion
13     }
14 }

```

```

1 //product
2 public interface Veicolo {
3     String guida();
4 }

```

```

1 // ConcreteProduct 1
2 public class Auto implements Veicolo {
3     @Override
4     public String guida() {
5         return "Guidando un'Auto: trasporto 4 persone
           .";
6     }
7 }
8
9 // ConcreteProduct 2
10 public class Camion implements Veicolo {
11     @Override
12     public String guida() {

```

```

13         return "Guidando un Camion: trasporto merce
14             pesante.";
15     }

```

CLIENT:

```

1 public class LogisticaClient {
2     public static void main(String[] args) {
3
4         // 1. Uso la Fabbrica di Auto
5         VeicoloFactory autoCreator = new AutoFactory
6             ();
7         String result1 = autoCreator.
8             pianificaLogistica();
9
10        System.out.println("Scenario 1: " + result1);
11        // Output: Scenario 1: Logistica pianificata.
12        //        Inizio: Guidando un'Auto: trasporto 4
13        //        persone.
14
15        // 2. Uso la Fabbrica di Camion
16        VeicoloFactory camionCreator = new
17            CamionFactory();
18        String result2 = camionCreator.
19            pianificaLogistica();
20
21        System.out.println("Scenario 2: " + result2);
22        // Output: Scenario 2: Logistica pianificata.
23        //        Inizio: Guidando un Camion: trasporto
24        //        merce pesante.
25    }
26 }

```

- **Caratteristiche:**

- Consente polimorfismo e late binding nella creazione (il tipo del veicolo si stabilisce a run time).
- I factory method non possono essere statici (se fossero statici le sottoclassi non potrebbero sovrascriverli).
- Detti anche **virtual constructors**.

- **Esempio:** applicazioni come Word/Excel:
 - Application (Creator) gestisce logica comune.
 - MyApplication (ConcreteCreator) implementa `createDocument()`.

Abstract Factory

- **Scopo:** creare **famiglie di oggetti compatibili** senza specificarne le classi concrete. (si posiziona a un gradino sopra rispetto a i factory method, serve quando possono esserci tipi diversi ma strutture simili, ad esempio macchina tesla o ford)

```

1 // Abstract Factory
2 interface GUIFactory {
3     auto createAuto();
4 }
5 class TeslaFactory implements GUIFactory {
6     public Auto createAuto() {
7         return new Tesla();
8     }
9 }

```

- **Problema:** serve creare oggetti correlati (es. elementi UI con stesso stile) garantendo coerenza tra di loro.
- **Soluzione:** introdurre **AbstractFactory** con un metodo fabbrica per ogni tipo di prodotto astratto.
- **Struttura:**
 - AbstractFactory: interfaccia con metodi per creare ogni AbstractProduct.
 - ConcreteFactory: implementa creazione di ConcreteProduct compatibili tra loro.
 - Product: interfaccia comune per gli oggetti creati.
 - ConcreteProduct: istanze concrete coerenti con la factory.
- **Esempio: GUI cross-platform**
 - GUIFactory con `createButton()`, `createCheckbox()`.
 - MacFactory → MacButton, MacCheckbox.
 - WinFactory → WinButton, WinCheckbox.
 - Il client conosce solo la factory adatta al sistema operativo e la usa per creare tutti i componenti.

7.12 Model View Controller (MVC)

Problema: una stessa informazione deve essere mostrata con viste differenti (es. colore: RGB, esadecimale, slider). Serve mantenere **coerenza tra viste** quando l'utente modifica i dati.

- **Soluzione ingenua:** viste che si aggiornano a vicenda → forte accoppiamento, difficile manutenzione. (se ho la lunghezza di una strada avrei una vista in metri e una in miglia)
- **Soluzione MVC:** separare *dati*, *presentazione* e *logica di interazione*.

Componenti principali:

- **Model:** contiene lo stato condiviso e gestisce i dati. Unico punto di verità.
- **View:** rappresenta l'interfaccia utente. Mostra i dati e riceve input.
- **Controller:** gestisce la logica applicativa e l'interazione. Riceve input dalla View e aggiorna il Model.

Ciclo di interazione tipico:

1. L'utente interagisce con una View → evento al Controller.
2. Il Controller elabora l'evento e (se necessario) modifica il Model.
3. Il Controller può aggiornare subito la View (es. messaggi d'errore).
4. Il Model cambia stato e **notifica tutte le View** registrate.
5. Le View aggiornano i dati mostrati recuperandoli dal Model (**politica pull**).

```
1  // Contratto Observer (La View lo implementera')
2  public interface Observer {
3      void update();
4  }
5
6  // Contratto Subject (Il Model lo implementera')
7  public interface Subject {
8      void registerObserver(Observer o);
9      void removeObserver(Observer o);
10     void notifyObservers();
11 }
```

```

1  //model
2  public class UserModel implements Subject {
3      private String nome;
4      private String email;
5      // Lista di tutte le View interessate (gli Observer)
6      private List<Observer> observers = new ArrayList<>();
7
8      public void loadUser(int userId) {
9          this.nome = "Mario Rossi";
10         this.email = "mario.rossi@example.com";
11         // Al caricamento, notifica per la
12         // visualizzazione iniziale
13         notifyObservers();
14     }
15
16     // Metodo chiave del Subject
17     @Override
18     public void registerObserver(Observer o) {
19         observers.add(o);
20     }
21
22     // Metodo chiave del Subject
23     @Override
24     public void notifyObservers() {
25         for (Observer observer : observers) {
26             observer.update(); // Il Model chiede a OGNI
27             // View di aggiornarsi
28         }
29     }
30
31     // Metodo di business logic, modificato per
32     // notificare
33     public void setEmail(String newEmail) {
34         this.email = newEmail;
35         // DOPO aver modificato lo stato, il Model
36         // notifica autonomamente.
37         notifyObservers(); // <--- Nessuna dipendenza dal
38         // Controller!
39     }
40
41     // Getters...
42     public String getNome() { return nome; }

```

```

38     public String getEmail() { return email; }
39 }

```

```

1  //view
2  public class UserView implements Observer {
3      private UserModel model; // Riferimento al Model per
4      prendere i dati
5      private UserController controller; // Riferimento al
6      Controller per inviare l'input
7
8      public UserView(UserModel model, UserController
9      controller) {
10         this.model = model;
11         this.controller = controller;
12         // Passo cruciale: la View si registra come
13         osservatore del Model
14         this.model.registerObserver(this);
15     }
16
17     // Metodo chiamato dal Model quando lo stato cambia (
18     la notifica)
19     @Override
20     public void update() {
21         // La View reagisce alla notifica chiamando i
22         getter del Model
23         String nome = model.getNome();
24         String email = model.getEmail();
25
26         System.out.println("--- View Aggiornata (Notifica
27         Ricevuta) ---");
28         System.out.println("Nome: " + nome);
29         System.out.println("Email NUOVA: " + email);
30         System.out.println("
31         -----");
32     }
33
34     // Metodo per inviare l'input al Controller
35     public void fireEmailChange(String newEmail) {
36         // La View parla SOLO con il Controller
37         controller.handleEmailChangeRequest(newEmail);
38     }
39 }

```

```

1  //contrller
2  public class UserController {
3      private UserModel model;
4      // La View e' usata solo per essere passata al Model,
5      // ma la sua presenza non e' piu' necessaria per l'
6      // aggiornamento.
7
8      public UserController(UserModel model) {
9          this.model = model;
10     }
11
12     // Si occupa solo di input: prende la richiesta e la
13     // passa al Model.
14     public void handleEmailChangeRequest(String newEmail)
15     {
16         System.out.println("Controller: Ricevuta
17         richiesta cambio email.");
18
19         // Il Controller NON chiama la View.
20         // L'unica sua responsabilita' e' AGGIORNARE IL
21         // MODEL.
22         model.setEmail(newEmail);
23
24         // Il Model fara' il resto grazie all'Observer
25         // pattern.
26     }
27 }

```

Vantaggi:

- Separazione netta tra interfaccia, dati e logica.
- Riutilizzo delle View con diversi Model.
- Indipendenza dalle modalità di rappresentazione.

Pattern coinvolti:

- **Observer:** le View sono Observer del Model (pull).
- **Observer (eventi GUI):** i Controller osservano gli eventi della View.
- **Strategy:** i Controller sono Strategy delle View (gestione dell'input).
- **Composite:** le View sono spesso composte da più componenti GUI.

Note:

- Il Controller funge da *Adapter* tra Model e View.
- Struttura circolare: View→Controller→Model→View.

Svantaggi:

- Dipendenza circolare → difficile testing e sviluppo incrementale.
- Testing delle GUI complesso (librerie diverse, eventi asincroni).
- View e Controller dipendono dalla libreria grafica (es. JavaFX) → bassa portabilità.

Utilizzo tipico: sviluppo di **GUI interattive**. **Limite:** testing complicato e forte dipendenza dalle interfacce grafiche.

7.13 Model View Presenter (MVP)

Scopo: eliminare la dipendenza diretta tra Model e View, facilitando il **testing indipendente**.

Differenza chiave: il **Presenter** sostituisce il Controller come *intermediario completo* tra View e Model.

Flusso:

1. La View invia input al Presenter.
2. Il Presenter elabora l'input e aggiorna il Model.
3. Il Model notifica il Presenter (non più la View).
4. Il Presenter aggiorna la propria View.

```
1  //model
2  public class CounterModel {
3      private int count = 0;
4      private CountObserver presenter; // Ora notifica solo
           il Presenter
5
6      public void setObserver(CountObserver p) {
7          this.presenter = p;
8      }
9
10     public void increment() {
```

```

11         count++;
12         // Il Model notifica il Presenter, NON la View.
13         if (presenter != null) {
14             presenter.onCountChanged(count);
15         }
16     }
17
18     public int getCount() {
19         return count;
20     }
21 }

```

```

1 //view
2 public class ConsoleCountView implements CountView {
3     private CountPresenter presenter;
4
5     // La View deve conoscere il suo Presenter
6     public ConsoleCountView(CountPresenter presenter) {
7         this.presenter = presenter;
8     }
9
10    // Metodo chiamato dal Presenter per disegnare a
11    schermo
12    @Override
13    public void displayCount(String countValue) {
14        System.out.println("VIEW: Valore attuale del
15        Contatore: " + countValue);
16    }
17
18    // Simula l'azione dell'utente (Click sul pulsante +)
19    public void userClickedIncrement() {
20        // La View delega immediatamente la richiesta al
21        Presenter.
22        presenter.handleIncrementRequest();
23    }
24 }

```

```

1 // Contratto per i Presenter che osservano il Model
2 public interface CountObserver {
3     void onCountChanged(int newCount);
4 }
5 //Presenter
6 public class CountPresenter implements CountObserver {

```

```

7     private CounterModel model;
8     private CountView view;
9
10    public CountPresenter(CounterModel model, CountView
        view) {
11        this.model = model;
12        this.view = view;
13
14        // 1. Il Presenter si registra come Observer del
            Model
15        this.model.addObserver(this);
16    }
17
18    // Logica di gestione dell'input (chiamata dalla View
        )
19    public void handleIncrementRequest() {
20        // 2. Il Presenter comanda il Model
21        model.increment();
22        // NON c'e' bisogno di chiamare view.displayCount
            qui.
23    }
24
25    // Logica di reazione al Model (il metodo dell'
        Observer)
26    @Override
27    public void onCountChanged(int newCount) {
28        // 3. Il Presenter riceve la notifica dal Model
29        // 4. Il Presenter decide come presentare il dato
            e comanda la View
30        String formattedCount = String.valueOf(newCount);
31        // Logica di presentazione
32        view.displayCount(formattedCount);
33    }

```

Effetti:

- Model e View non comunicano mai direttamente.
- Il Presenter diventa un **Adapter bidirezionale**.
- Si può testare ogni componente isolatamente (Design for Testing).

Attenzione:

- Evitare **reference escaping** tra Model e Presenter (il model non ha possibilità di modificare direttamente il model (se non con i metodi set)).
- Mantenere separazione rigorosa di responsabilità.

Risultato: sistema più testabile, meno dipendente dall'interfaccia, più modulare.

7.14 Builder

Problema: inizializzare oggetti con molti parametri (obbligatori + opzionali) in modo leggibile, sicuro e coerente.

7.14.1 Telescoping Constructor Pattern

Idea: più costruttori con combinazioni diverse di parametri opzionali. **Problemi:**

- Numero di costruttori cresce esponenzialmente.
- Ambiguità con parametri dello stesso tipo (non posso avere due costruttori uno dove inizializzo solo A e l'altro solo B se entrambi sono dello stesso tipo).

7.14.2 JavaBeans Pattern

Idea: costruttore con soli parametri obbligatori + setter per opzionali. **Problemi:**

- Oggetti non immutabili (richiede setter → campi non final).
- Possibili stati intermedi non consistenti.

7.14.3 Builder Pattern

Idea: combinare vantaggi di entrambi:

- Classe interna **Builder** con stessi campi della classe esterna.
- **Builder** ha costruttore per parametri obbligatori e setter fluenti per opzionali.
- Metodo `build()` crea l'istanza finale.

```

1 public class MyClass {
2     private final T0 optionalField1;
3     private final T1 mandatoryField;
4     private final T2 optionalField2;
5
6     private MyClass(Builder builder) {
7         mandatoryField = builder.mandatoryField;
8         optionalField1 = builder.optionalField1;
9         optionalField2 = builder.optionalField2;
10    }
11
12    public static class Builder {
13        private T1 mandatoryField;
14        private T0 optionalField1 = defaultValue1;
15        private T2 optionalField2 = defaultValue2;
16
17        public Builder(T1 mf) {
18            mandatoryField = mf;
19        }
20
21        public Builder withOptionalField1(T0 of) {
22            optionalField1 = of;
23            return this;
24        }
25
26        public Builder withOptionalField2(T2 of) {
27            optionalField2 = of;
28            return this;
29        }
30
31        public MyClass build() {
32            return new MyClass(this);
33        }
34    }
35 }

```

Vantaggi:

- Oggetti **immutabili** (campi final).
- Nessuno stato intermedio inconsistente.
- Notazione fluente e leggibile.

- Costruzione thread-safe e sicura.

Nota: la classe interna `Builder` è **statica**, quindi può essere istanziata senza un oggetto esterno.

Sintassi tipica (notare che i vari costruttori builder ritornano se stessi, quindi posso concatenare più costruttori):

```
1 MyClass x = new MyClass.Builder(mandatory)
2     .withOpt1(o1)
3     .withOpt2(o2)
4     .build();
```

Conclusione: pattern creazionale che consente costruzione modulare, sicura e leggibile di oggetti complessi.

8 UML

linguaggio che ha come scopo quello di creare degli standard per la rappresentazione visuale di un software

8.1 Analisi del testo naturale

8.1.1 Organizzazione del design

ci sono tre modi principali per iniziare il flusso di lavoro

- **Pattern:** riconoscere situazioni comuni a partire dai dati.
- **TDD (Test Driven Design):** partire dalla soluzione più semplice, definendo classi solo all'occorrenza.
- **Noun Extraction:** estrazione dei sostantivi dalle specifiche per identificare classi e relazioni.

8.1.2 Noun Extraction

per identificarli:

1. Identificare sostantivi/frasi nominali nelle specifiche.
2. Sfoltire secondo criteri (ridondanza, vaghezza, eventi, metalinguaggio, concetti esterni, attributi).
3. Individuare relazioni e costruire gerarchie di classi.

8.1.3 Criteri di sfoltimento

- **Ridondanza:** sinonimi o espressioni equivalenti (*library member* vs *member of library*).
- **Vaghezza:** nomi generici (es. *items*).
- **Eventi/operazioni:** non rappresentano stati (es. *loan*).
- **Metalinguaggio:** elementi logici del programma (*system*, *rules*).
- **Esterne:** concetti fuori dal controllo del sistema (*library*, *week*).
- **Attributi:** dati primitivi (*name of member*).

8.1.4 Relazioni tra classi

- Collegare classi con linee (non frecce), poi definire cardinalità UML.
- Cercare generalizzazioni e fattorizzazioni:
 - *StaffMember* è un tipo di *LibraryMember*.
 - *Item* generalizza *CopyOfBook* e *Journal*.
- Vincoli esprimibili con OCL (Object Constraint Language) (alcuni dettagli non si possono descrivere a grafico e necessitano di altri modi).

8.1.5 Stato: concreto vs astratto

bisogna anche fare in modo che lo stato della classe sia rappresentabile al meglio:

- **Stato concreto:** combinazione dei valori di tutti gli attributi (ogni singolo libro con le sue proprietà (titolo, autore, codice, posizione. . .)).
- **Stato astratto:** rappresentazione compatta e significativa di stati concreti (condizione logica generale (se il codice inizia per 'L', il libro è "solo lettura in sala")). lo stato astratto contiene più stati concreti ed è un concetto (es: temp da 0.000001 a 18 gradi, freddo)

8.1.6 Casi particolari

- **Stateless object:** senza attributi né stato (oggetti che rappresentano astrazioni funzionali).
- **Oggetti immutabili:** un solo stato possibile (es. oggetto *String*).

8.2 State Diagram

8.2.1 Concetto e struttura

- Deriva dagli *State Charts*.
- Automa: $\langle S, I, U; \delta, t, s_0 \rangle$.
 - S: insieme finito e non vuoto degli stati;
 - I: insieme finito dei possibili input;
 - U: insieme finito dei possibili output;
 - δ : funzione di transizione;

- t : funzione di uscita;
- s_0 : stato iniziale.
- $\delta : S \times I \rightarrow S$ (transizione), s_0 stato iniziale.

8.2.2 Esempio: flip-flop di Moore

- Stati: 0 e 1.
- Ingressi: S (Set), R (Reset).
- Uscita: stato corrente.

8.2.3 Automi di Mealy

$$t : S \times I \rightarrow U$$

L'uscita dipende dallo stato e dall'ingresso, negli automi di moore invece è lo stato che dipende da S ed I questo cambia perché se stessimo sempre nello stesso stato in moore avremmo lo stesso output in mealy no.

8.3 In UML

per rappresentare questi diagrammi in uml:

- Ogni stato: rettangolo; stato iniziale: pallino nero.
- Complessità eccessiva $\Rightarrow$ troppe responsabilità $\Rightarrow$ decomporre la classe.

8.3.1 Azioni e eventi

- **Evento:** metodo che provoca transizione.
- **Azione:** effetto conseguente (invocazione su altri oggetti).

8.3.2 Guardie

- Disambiguano transizioni dallo stesso stato (un libro che viene preso in prestito, il sistema potrebbe andare nello stato non disponibile nel caso in cui quella fosse l'ultima copia, altrimenti resterebbe nello stato disponibile).

8.3.3 Eventi speciali

- **Time event:** eventi temporizzati, `after(duration)` (indica tempo massimo di permanenza nello stato).
- **Change event:** `when(condition)` (una sorta di trigger, eventi innescati da un cambiamento).

8.4 Superstate

- Raggruppa stati comuni, elimina ridondanze (semaforo o spento o acceso, se acceso a sua volta diventa un altro diagramma degli stati).
- **History state:** possiamo anche avere un diagramma che memorizza lo stato precedente (es: semaforo si accende nel verde? per gli incroci).
- **Region:** possono anche rappresentare concorrenza (se devo fare più cose es: ogni 5 input scrivi qualcosa)(stati concorrenti divisi da linee tratteggiate).

8.5 Class Diagram

8.5.1 Concetto e struttura

- Vista statica del software: classi, metodi, attributi, relazioni.
- **oggetti:** Classe, **I:** Interfaccia, **A:** Astratta, **E:** Enum.
- **metodi:** preceduti da un cerchio e tipo di valore di ritorno
- **attributi:** preceduti da un quadrato
- **relazioni:** frecce che connettono gli oggetti

8.5.2 Regole grafiche

- i metodi e attributi possono essere privati `protected` o pubblici (rossi gialli verdi)
- Corsivo: elemento astratto.
- Sottolineato: statico.
- « »: stereotipo (parola per identificare un oggetto).

8.5.3 Relazioni UML

- **Dipendenza (tratteggiata):** A dipende da B.
- **Associazione (linea):** legame bidirezionale (senza freccia) o monodirezionale (con freccia).
- **Aggregazione (rombo bianco):** “ha un” — relazione più forte rispetto ad associazione (sia gli studenti che il professore rimangono tali anche se non sono associati l’uno all’altro, in questo caso invece c’è una relazione più stretta).
- **Composizione (rombo nero):** vita legata al contenitore (L’oggetto contenuto non può nascere prima che nasca il contenitore e non può morire dopo che muore il contenitore).
- **Implementazione (triangolo):** una classe può implementare un’interfaccia.

8.6 Sequence Diagram

8.6.1 Concetto e struttura

- Rappresenta il flusso temporale delle invocazioni.
- Elementi: attori, invocazioni, ritorni, cicli (*loop*), condizioni (*opt*).

8.7 Activity Diagram

8.7.1 Concetto e struttura

- Stati $\rightarrow$ attività.
- Transizioni implicite.
- simili a state diagram.

8.7.2 Livelli di astrazione

- Logica interna di processi.
- Flusso di un metodo.
- Dettaglio di un caso d’uso.

8.7.3 Sincronizzazione

- Barre JOIN/FORK (AND o OR).
- Inizio e fine obbligatori.

8.7.4 Decisioni

- Archi mutualmente esclusivi.
- Copertura completa delle condizioni.
- Differiscono dalle guardie (decisione esterna, non di sistema).

8.7.5 Swim Lane

- Divisione per responsabilità (corsie verticali).

8.8 Use Case Diagram

8.8.1 Concetto e struttura

- Rappresentano interazioni attore–sistema.
- Linguaggio semplice, orientato al cliente.

8.8.2 Componenti

- Attori, Casi d'uso, Relazioni.
- Ogni attore $\leftrightarrow$ almeno un caso d'uso.

8.8.3 Relazioni

- **Include:** fattorizzazione comune.
- **Extend:** comportamento eccezionale.
- **Generalizzazione:** ereditarietà tra attori o use cases.

8.9 Component Diagram

8.9.1 Concetto e struttura

- Mostra e raggruppa i componenti del sistema.
- Componenti = rettangoli, interfacce = cerchi.
- Stereotipi: etichette tra « ».

8.9.2 Identificazione componenti

- Parti rimpiazzabili/versionabili.
- Funzioni ben determinate.
- Struttura gerarchica.

8.10 Deployment Diagram

- Mostra la dislocazione fisica delle risorse a run-time.
- Nodi = macchine fisiche; collegamenti = protocolli (HTTP, RMI, ecc.).
- Utile per ottimizzare il deploy e la distribuzione delle risorse.

Tabella Riassuntiva dei Diagrammi UML

Diagramma	Focus e Obiettivo	Domande a cui Risponde
Use Case	Mostra i requisiti funzionali del sistema: cosa fa il sistema in relazione agli attori esterni.	Chi usa il sistema? Quali sono le principali funzionalità offerte?
Activity	Modella il flusso logico (sequenza di azioni, decisioni, cicli) per completare un processo o un caso d'uso specifico.	Come si svolge un processo dall'inizio alla fine? Quali sono le decisioni e le diramazioni possibili?
Sequence	Modella l' ordine cronologico dei messaggi scambiati tra gli oggetti per raggiungere un obiettivo.	In quale sequenza i messaggi passano tra gli oggetti? Qual è l'ordine cronologico degli eventi?
Component	Modella l'organizzazione e le dipendenze tra i componenti software (moduli, librerie, servizi) del sistema.	Come è suddiviso il codice a livello logico? Quali componenti dipendono da altri?
Deployment	Modella la disposizione fisica dei componenti software sui nodi hardware (server, dispositivi) in fase di esecuzione.	Su quali server o dispositivi fisici viene eseguito ogni componente? Come sono collegati i nodi hardware?

9 Mocking

9.0.1 nomenclatura:

- **SUT:** ciò che viene testato (system under test)
- **DOC:** il componente da cui l'oggetto testato dipende (dependent-on component)

9.1 Struttura di un Test

Ogni test deve essere chiaro e leggibile, seguendo 4 fasi:

1. **Set Up:** inizializzazione del SUT e dei Test Double.
2. **Exercise:** esecuzione del codice da testare.
3. **Verify:** controllo dei risultati (asserzioni).
4. **Teardown:** pulizia e ripristino ambiente.

Ogni test deve verificare una singola funzionalità per aumentare la localizzazione delle anomalie.

9.2 Test Double

Termine generico che identifica un qualunque oggetto che sostituisce un componente reale (DOC) durante il test.

Serve per:

- simulare componenti non disponibili o con effetti collaterali;
- controllare input/output del SUT;
- forzare scenari complessi o randomici.

Tipi principali:

- **Dummy:** passato come parametro ma mai usato.
- **Stub:** fornisce risposte predefinite a input indiretti.
- **Mock:** fa quello che fa uno stub ma inoltre, tiene traccia di quali metodi il SUT ha chiamato e con quali valori, il tutto per verificare che il comportamento del SUT sia corretto.
- **Spy:** a diff del mock gli spy partono da un oggetto reale, creano un oggetto spy da esso per osservare le chiamate che fa il SUT.
- **Fake:** implementazione reale ma semplificata (e.g. DB in memoria).

9.3 Regole per il Mocking

- Un solo oggetto reale per test (`new` unica).
- Tutti i DOC devono essere mockati (non ci devono essere componenti reali che rallentano).
- Non mockare classi delle librerie standard (e.g. `List`).
- Possibile eseguire **spy** parziale di un oggetto reale.

9.4 Mockito

Framework open source per la creazione e gestione di Test Double. Basato su Reflection, migliora leggibilità e controllo.

Creazione di Test Double

- `mock(Class.class)`: crea dummy, stub o mock con implementazione minima.
- `spy(obj)`: crea uno spy da un oggetto reale (chiama metodi reali se non stubbati).

Stubbing Definisce il comportamento dei metodi mockati.

- `when(mock.method(args)).thenReturn(value)`
dice che se chiamato quel metodo deve restituire sempre value
- `thenThrow()`, `thenAnswer()`, `thenCallRealMethod()`
stessa cosa, deve restituire un'eccezione/una srisposta ecc..
- `doReturn()/doThrow()/doNothing()`
usati per metodi void.

Verifica delle Chiamate `verify(mock, howMany).method(args)`, spesso si desidera verificare non solo i valori di ritorno ma anche quante volte un metodo è stato chiamato:

- `times(n)`, `never()`, `atLeast(n)`, `atMost(n)`
- `InOrder in0 = inOrder(mock1, mock2, ...)` per verificare l'ordine.

esempio di verifica che sia stato chiamato il metodo doSomething con un determinato oggetto (a volte complesso quindi si usa argumentCaptor)

Cattura Parametri

```
1      ArgumentCaptor<Person> arg = ArgumentCaptor.  
        forClass(Person.class);  
2      verify(mock).doSomething(arg.capture()); //  
        intrappola in arg l'oggetto chiamato in input  
3      assertEquals("John", arg.getValue().getName());  
        // si verifica l'oggetto passato  
4  
5      // si usa verify e non when perché non stiamo  
        definendo un comportamento ma lo stiamo  
        verificando
```

Argument Matchers Consentono di evitare la specifica di valori esatti:

- any(), anyInt(), anyString()
- eq(value) per uguaglianza specifica
- argThat(...) per condizioni personalizzate

Reset reset(mock) ripristina lo stato originale del Test Double.

9.5 Esempio: Pattern Observer (Pull)

```
1      @Test  
2      void modelTest() {  
3          Model model = new Model();  
4          Observer obs = mock(Observer.class);  
5          Observer obs1 = mock(Observer.class);  
6          model.addObserver(obs);  
7          model.addObserver(obs1);  
8          model.setTemp(42.0, scale);  
9          verify(obs).update(eq(model), eq("42.0"));  
10         verify(obs1).update(eq(model), eq("42.0"));  
11     }
```

9.6 Injection con Mockito

si possono iniettare dentro oggetti reali oggetti mock basta che all'inizio mettiamo la notazione `@ExtendWith(MockitoExtension.class)`

```
1      @ExtendWith(MockitoExtension.class)
2      class TestPartita {
3          @Mock Tavolo mockedTable;
4          @InjectMocks Partita SUT;
5
6          @Test
7          void test() {
8              when(mockedTable.inMostra(Card.get(...)))
9                  .thenReturn(true);
10             assertThat(SUT.
11                 controllaSeCartaPresenteSuTavolo(...))
12                 .isTrue();
13         }
14     }
```

Regole:

- `@Mock`: oggetti mockati.
- `@InjectMocks`: SUT reale in cui vengono iniettati i mock.
- Iniezione tramite costruttore, setter o se non esistono direttamente nell'attributo privato (basta che non sia final) es: inject di tavolo in Partita.
- Attributi `final` non modificabili $\Rightarrow$ usare **Mocked Construction**.

9.7 Mocked Construction

Permette di mockare oggetti al momento della loro creazione (al try bisogna passare una funzione lambda come parametro per evitare stati inconsistenti, infatti se non usassi la lambda dovrei invocare l'oggetto mockato per definire i suoi comportamenti causando inconsistenza).

```
1      @Test
2      void constructorTest() {
3          try (MockedConstruction<MultiMazzo> mc =
4              mockConstruction(MultiMazzo.class, (mock, ctx) -> {
5                  when(mock.draw()).thenReturn(Card.get("2C"));
6              })) {
```

```

7      Mazziere m = new Mazziere();
8      assertThat(m.draw()).isEqualTo("2C");
9  }
10 }

```

- Mocka ogni `new MultiMazzo()`.
- Permette stubbing al momento della creazione.
- Verifiche via `mocked.constructed().get(0)`.

9.8 Mocking di Iterable

se si usasse questa implementazione, la prima volta che viene richiamata viene restituito l'el giusto, dalla seconda in poi, si deve ricreare l'iteratore (in java l'iteratore è usa e getta) e quindi fallirebbe.

con `thenReturn` → restituisci sempre lo stesso iteratore → funziona solo la prima volta;

con `thenAnswer` → viene ricreato a ogni chiamata, risolvendo il problema.

```

1 Iterable<Integer> mockIterable = mock(Iterable.class);
2 when(mockIterable.iterator()).thenReturn(List.of(1, 2, 3)
   .iterator());

```

Serve per oggetti che implementano `Iterable`.

```

1 public static <T> void whenIterated(Iterable<T> p, T... d
   ) {
2     when(p.iterator()).thenAnswer(inv -> List.of(d).
       iterator());
3 }

```

Esempio:

```

1 MockUtils.whenIterated(partita, ritorno[]); // partita e'
   il mock su cui si definisce il comportamento, ritorno
   sono gli oggetti su cui itera

```

Garantisce che ogni iterazione crei un nuovo iteratore valido.

9.9 Riepilogo Tipi di Test Double

Tipo	Descrizione
Dummy	Solo segnaposto, mai usato.
Stub	Restituisce valori predefiniti.
Mock	Verifica interazioni/output indiretti.
Spy	Controlla chiamate su oggetto reale.
Fake	Implementazione semplificata non di produzione.

10 Verifica e Convalida

10.1 Verifica e Convalida

10.1.1 Terminologia

- **Verifica**: confronto del software con le **specifiche formali** (analista).
- **Convalida**: confronto del software con i **requisiti informali** (committente).
- **Differenze principali**:
 - Requisiti: linguaggio vicino al dominio del committente (informali).
 - Specifiche: linguaggio formale vicino allo sviluppatore.
 - I requisiti possono cambiare, le specifiche sono più stabili.
- Verifica $\Rightarrow$ test su specifiche (più semplici da scrivere).
- Convalida $\Rightarrow$ test sui requisiti (più complessi da formalizzare).
- Obiettivo comune: individuare errori.

Terminologia IEEE :

abbiamo 3 termini da definire rigorosamente:

10.1.2 Malfunzionamento (Failure)

- Deviazione dal comportamento corretto osservabile dall'esterno.
- Non visibile nel codice ma nel comportamento.
- Può riguardare specifiche (verifica) o requisiti (convalida).
- **Esempio**:

```
1      static int raddoppia(int par) {  
2          int risultato;  
3          risultato = (par * par);  
4          return risultato;  
5      }  
6  // Input: 3, Output: 9  implica malfunzionamento
```

10.1.3 Difetto (Fault/Anomalia)

- Punto del codice che causa un malfunzionamento.
- **Condizione necessaria ma non sufficiente** per un failure.
- Esempio: nell'esempio precedente, il difetto è `risultato = (par * par);`.
- Non sempre causa failure (es. argomento 0 o 2).
- Possibili compensazioni di anomalie (errori che si annullano).

10.1.4 Sbaglio (Mistake)

- Causa di un'anomalia $\rightarrow$ errore umano.
- Tipi:
 - Errore di battitura (`*` invece di `+`).
 - Errore concettuale (non comprendere “raddoppiare”).
 - Errore di linguaggio (confondere gli operatori).

10.1.5 Caso Ariane 5

- **Data:** 4 giugno 1996.
- **Evento:** esplosione del razzo per failure software.
- **Malfunzionamento:** overflow durante conversione `float 64` $\rightarrow$ `int 16`.
- **Difetto:** mancata gestione dell'eccezione di overflow.
- **Sbaglio:** riuso di codice da Ariane 4, dove l'overflow non poteva verificarsi.
- **Lezione:** analizzare sempre i limiti di validità del software in contesti diversi.

10.2 Tecniche di Verifica e Convalida

10.2.1 Classificazione generale

- **Tecniche statiche:** analisi sintattica del codice.
 - Esempi: metodi formali, analisi dataflow, modelli statistici.
- **Tecniche dinamiche:** analisi tramite esecuzione.

- Esempi: testing, debugging.
- Statiche = più veloci e complete, ma difficili da definire.
- Dinamiche = più facili da creare, ma coprono stati finiti del programma.

10.2.2 Piramide concettuale

- **Vertice ideale:** verifica perfetta (prova logica o ricerca esaustiva).
- **Basi della piramide:**
 - **Semplificazione eccessiva degli obiettivi** (sinistra): se vuoi verificare che un puntatore punti all’oggetto giusto, non basta verificare che non sia null.
 - **Inaccuratezza pessimistica** (centro): “Se non dimostro l’assenza di un problema, assumo che esista.”
 - **Inaccuratezza ottimistica** (destra): “Se non trovo problemi, assumo che non ci siano.”
- Le tecniche reali si collocano tra questi estremi.

10.2.3 Metodi Formali

- Scopo: dimostrare l’assenza di anomalie nel prodotto finale (Provano matematicamente che il programma non può mai sbagliare).
- Esempi:
 - Analisi del dataflow.
 - Dimostrazione di correttezza logica.
- Segue l’approccio dell’inaccuratezza pessimistica.

10.2.4 Testing

- Scopo: rilevare malfunzionamenti (Eseguono il programma e verifico che si comporti correttamente su molti casi significativi.)
- Aumenta la fiducia sull’affidabilità (non dimostra la correttezza).
- Tipi:
 - **White box:** accesso completo al codice.

- **Black box:** test su interfacce, senza vedere il codice.
- **Gray box:** conoscenza parziale (es. pattern MVC).
- Segue la logica dell'inaccuratezza ottimistica.
- **TDD:** scrittura dei test prima del codice $\Rightarrow$ approccio black box.

10.2.5 Debugging

- Scopo: localizzare l'anomalia che causa un malfunzionamento noto.
- Richiede un failure riproducibile.
- **Differenze col testing:**
 - Testing: cerca malfunzionamenti.
 - Debugging: individua la causa.
- Approcci:
 1. Analisi passo-passo degli stati intermedi.
 2. Divide et impera: analisi sezionale con breakpoint o print.

10.3 Testing

10.3.1 Definizioni

10.4 Legenda

:

- $D \rightarrow$ dominio
- $R \rightarrow$ codominio
- $P(d) \rightarrow$ esecuzione di un programma su un certo input d appartenente a D
- $ok(P, D) \rightarrow P$ corretto su qualsiasi d di D
- **Programma Corretto:**

$$P \text{ è corretto} \Leftrightarrow \forall d \in D, ok(P, d)$$

Definizione di test

- Test $T \subseteq D$, caso di test $t \in T$.
- Esecuzione: $\forall t \in T$, eseguire $P(t)$ e confrontare risultato atteso.
- P supera T se:

$$ok(P, T) \Leftrightarrow \forall t \in T, ok(P, t)$$

- Un test ha successo se rileva malfunzionamenti:

$$successo(T, P) \Leftrightarrow \exists t \in T : \neg ok(P, t)$$

Test ideale

- T ideale per P se:

$$ok(P, T) \Rightarrow ok(P, D)$$

- **Tesi di Dijkstra:** il test può mostrare la presenza di errori ma non dimostrarne l'assenza.
- Il test esaustivo $T = D$ è impraticabile (es. 2^{64} combinazioni $\Rightarrow > 600$ anni a 1ns/test).

10.4.1 Proprietà

$C \rightarrow$ criterio secondo cui si sceglie quali test fare

Affidabilità :

un criterio è affidabile se tutti i test effettuati su P o hanno tutti successo o nessuno:

$$affidabile(C, P) \Leftrightarrow (\forall T_1, T_2 \in C) [successo(T_1, P) \Leftrightarrow successo(T_2, P)]$$

Validità :

un criterio è valido se qualora P non sia corretto, esiste per forza almeno un test che ha successo (individua l'anomalia):

$$valido(C, P) \Leftrightarrow (\neg ok(P, D) \Rightarrow \exists T \in C \mid successo(T, P))$$

Esempio

```
1 static int raddoppia(int par) {  
2     int risultato;  
3     risultato = (par * par);  
4     return risultato;  
5 }
```

- $T = \{0, 2\}$: affidabile ma non valido.
- $T = \{0, 1, 2, 3, 4\}$: valido ma non affidabile.
- $T = \{d > 18\}$: valido e affidabile (i test hanno tutti successo e il programma fallisce sempre).

Conclusione

- Criterio valido $\wedge$ affidabile $\Rightarrow$ test ideale su qualsiasi programma (impossibile per Dijkstra).
- Formalmente:

$$affidabile(C, P) \wedge valido(C, P) \wedge T \in C \wedge \neg successo(T, P) \Rightarrow ok(P, D)$$

10.4.2 Utilità di un test

il compromesso sta nell'individuare i test utili:

- Caso di test utile se:
 1. Esegue il comando con l'anomalia.
 2. Porta a stato scorretto/inconsistente.
 3. Propaga l'inconsistenza all'output.
- Criterio di selezione $\Rightarrow$ quanti/quali casi di test includere.
- Ogni criterio ha una **metrica di copertura** $\Rightarrow$ misura la bontà del test.

10.4.3 Criteri di selezione

- Obiettivo: approssimare un test ideale, ogni criterio cerca di massimizzare la copertura del programma.
- Tipologie principali:

1. Copertura dei comandi
2. Copertura delle decisioni
3. Copertura delle condizioni
4. Copertura decisioni+condizioni
5. Copertura condizioni composte
6. Copertura condizioni e decisioni modificate (MCDC)

```
1      01  void main(){
2      02      float x, y;
3      03      read(x);
4      04      read(y);
5      05      if (x != 0)
6      06          x = x + 10;
7      07      y = y / x;
8      08      write(x);
9      09      write(y);
10     10  }
```

Copertura dei comandi

- comando = unità logica (non serve che vengano percorse tutte le unità di un if ma basta che l'if sia percorso almeno una volta), anche assegnazione è un comando.
- Ogni comando eseguibile deve essere eseguito almeno una volta.
- Metrica: $\frac{\text{comandi eseguiti}}{\text{comandi totali}}$.
- Non garantisce rilevazione errori (es. divisione per 0 non rilevata).

Copertura delle decisioni

- Ogni decisione resa sia vera che falsa da almeno un caso di test (cioè percorrere tutte le vie percorribili).
- Metrica: $\frac{\text{decisioni rese V/F}}{\text{decisioni totali}}$.
- **Implica** la copertura dei comandi.

Copertura delle condizioni

- Ogni condizione atomica resa sia vera che falsa.
- Non implica nessun criterio precedente.
- Decisione composta: le singole condizioni possono essere coperte senza esplorare tutti i rami.
- es: $x == 0$ et $y == 2$ (2 test: una volta soddisfa $x == 0$, una volta soddisfa $y == 2$, ma in alcuni casi non copre tutte le vie percorribili)

Copertura decisioni e condizioni

- Ogni decisione e ogni condizione atomica rese sia vere che false.
- **Implica** copertura decisioni e condizioni $\Rightarrow$ comandi.
- es: $x == 0$ et $y == 2$ (2 test: prima falsa, seconda vera prima vera seconda falsa, in entrambi i test nonostante si abbia rispettato il criterio si ha un esito falso)

Copertura delle condizioni composte

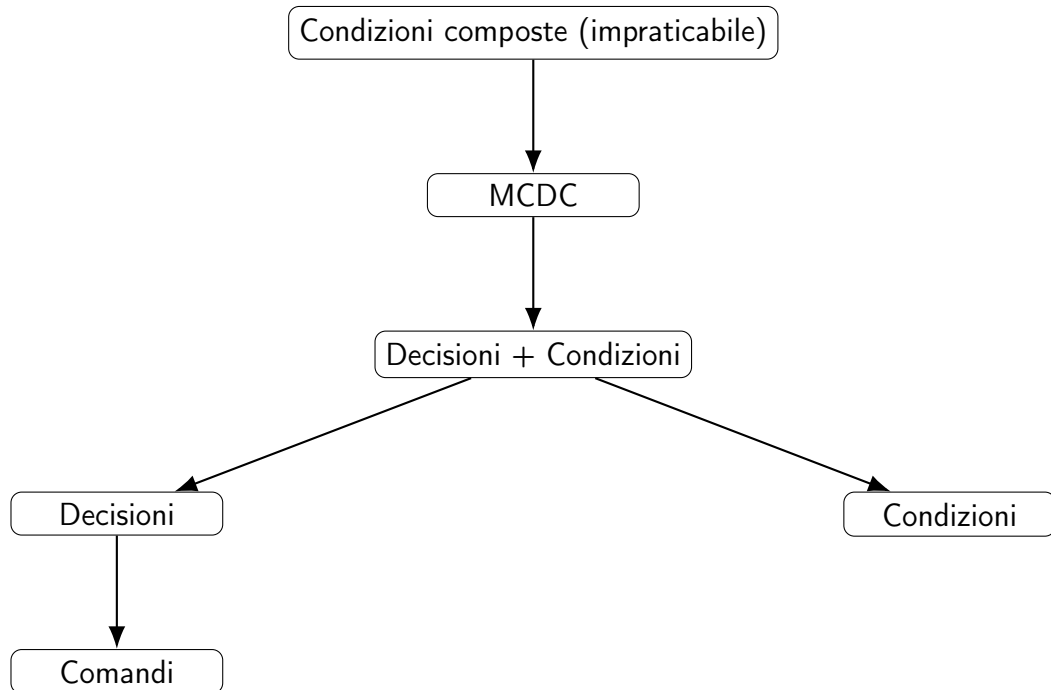
- Tutte le combinazioni V/F delle condizioni base.
- Crescita combinatoria (2^n combinazioni).
- **Implica tutti** i criteri precedenti ma è impraticabile.

Copertura condizioni e decisioni modificate (MCDC)

- Ogni condizione base deve poter modificare l'esito della decisione.
- Bastano $N + 1$ casi di test per N condizioni base.
- **Implica** copertura decisioni+condizioni.

1	Test	A	B	C	Risultato	Decisione
2	t1	v	v	v	v	caso base vero
3	t2	f	v	v	f	influenza di a
4	t3	v	f	v	f	influenza di b
5	t4	v	v	f	f	influenza di c

10.4.4 Implicazioni tra criteri di copertura



10.4.5 Altri criteri (cicli)

- Criteri precedenti non considerano iterazioni.
- Errori spesso emergono dopo più iterazioni.
- Necessari criteri che testino i cicli.

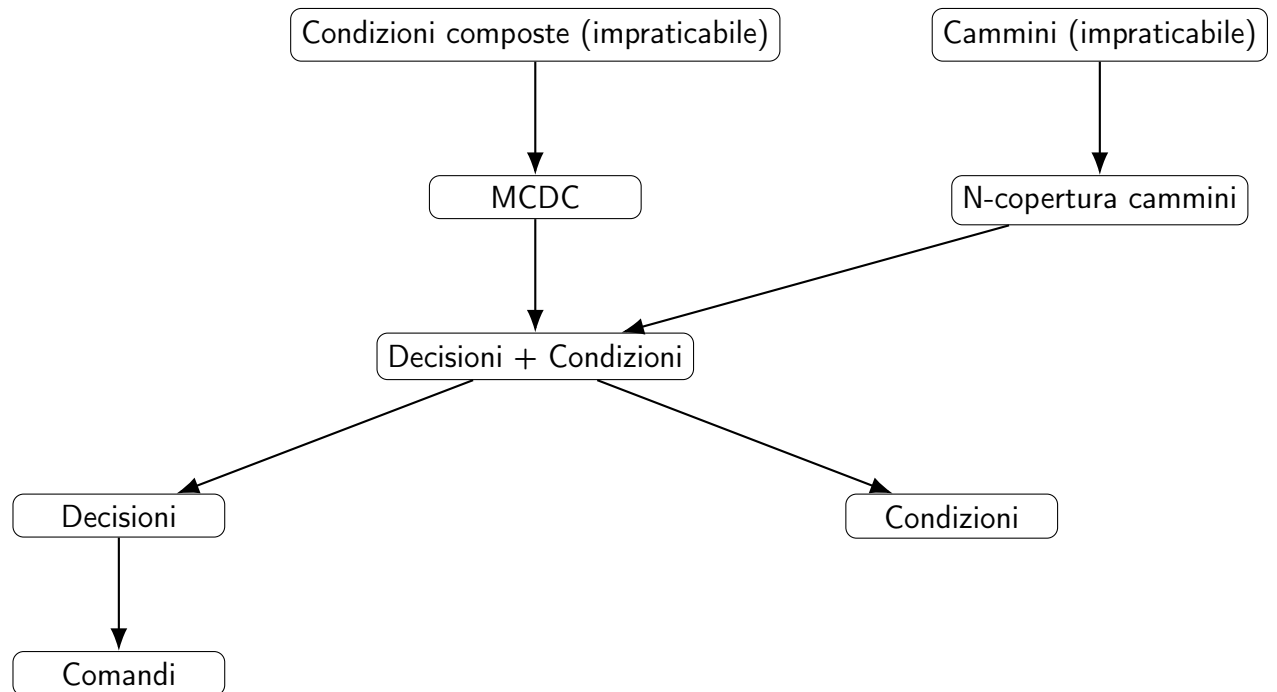
Copertura dei cammini

- Ogni cammino nel grafo di controllo deve essere percorso almeno una volta.
- Impraticabile con cicli (∞ cammini).

n-Copertura dei cicli

- Ogni cammino con $\leq n$ iterazioni per ciclo deve essere percorso.
- n indice del ciclo con **n=2** è ottima (testa: 0, 1, >1 iterazioni) (casi limite e caso intermedio).
- **Applicabile a programmi reali.**

10.4.6 Mappa finale delle implicazioni



11 Analisi Statica

11.1 Definizione generale

- Non esegue il programma: analizza il **codice sorgente** (insieme finito di istruzioni).
- Contrapposta all'**analisi dinamica**, che considera gli stati d'esecuzione (insieme infinito).
- Più economica del testing; non rileva errori legati ai valori delle variabili a runtime.
- Può individuare errori concettuali e potenziali anomalie.

11.2 Principali strumenti

1. Compilatori
2. Analisi del flusso dei dati (Data Flow)
3. Testing guidato da analisi statica
4. Criteri di copertura

11.3 Compilatori

Eseguono analisi statica preliminare prima della traduzione:

- **Analisi lessicale:** individua token errati.

```
1      dar ciao = 12; // dar sbagliato
```

- **Analisi sintattica:** controlla la struttura grammaticale (l'ordine delle parole non la correttezza delle stesse, se hanno senso messe insieme in quel modo).

```
1      int ciao = 12 // manca il punto e virgola
```

- **Controllo dei tipi:** verifica compatibilità dei tipi.

```
1      int ciao = 12;  
2      ciao += "hola"; // tipo errato
```

- **Analisi del flusso dei dati:** trova codice non raggiungibile o valori incoerenti.

```

1      if 1 != 1 {
2          int a = 1; //irraggiungibile
3      }

```

11.4 Analisi Data Flow

Obiettivo: studiare come le variabili vengono definite, usate e annullate nel codice. Ogni operazione appartiene a uno dei tre tipi:

- d:** definizione (assegnamento o modifica)
- u:** uso (lettura del valore)
- a:** annullamento (valore non più valido)

Regole fondamentali:

1. Ogni uso (**u**) deve essere preceduto da una definizione (**d**) senza annullamenti intermedi. $\Rightarrow$ Sequenza **a-u** = errore.
2. Ogni definizione deve essere seguita da almeno un uso prima di un nuovo d o a. $\Rightarrow$ Sequenze **d-a**, **d-d** = errore.
3. Ogni annullamento deve essere seguito da una definizione prima di un nuovo uso. $\Rightarrow$ Sequenza **a-a** = errore.

11.5 Esempio pratico

```

1 void swap(int &x1, int &x2) {
2     int x1;
3     x3 = x1;
4     x3 = x2;
5     x2 = x1;
6 }

```

Variabile	Sequenza	Anomalie
x1	a u u a	uso senza definizione
x2	d u d	nessuna
x3	d d d	definizioni multiple senza uso

x1: (riga 2 annulla l'assegnamento fatto in input (riga 1), 3,5 uso, uscita provoca annullamento)

11.6 Sequenze e cammini

Una **sequenza** di una variabile a secondo il cammino p è:

$$P(p, a) = \text{concatenazione delle operazioni su } a$$

Esempio:

```
1   01  void main() {
2   02      float a, b, x, y;
3   03      read(x);
4   04      read(y);
5   05      a = x;
6   06      b = y;
7   07      while (a != b)
8   08          if (a > b)
9   09              a = a - b;
10  10          else
11  11              b = b - a;
12  12      write(a);
13  13  }
```

$P([1,2,3,4,5,6,7,8,9,7,12,13], a) = a2, d5, u7, u8, u9, d9, u7, u12, a13$ (11 viene saltata poiché è nell'else e in questo cammino non lo consideriamo)

11.7 Espressioni regolari

I cammini del programma possono essere rappresentati come espressioni regolari:

- Simbolo | rappresenta **decisioni**.
- Simbolo * rappresenta **cicli**.

(GUARDA IMMAGINE SU APPUNTI PROF)

Limite: l'espressione regolare è una **astrazione pessimistica** → include cammini non percorribili.

11.8 Testing e analisi statica

- L'analisi statica aiuta a selezionare i casi di test.
- I criteri si basano su **sequenze definizione–uso (DU)** (ci consiglia di testare ogni definizione fino al suo uso).

11.9 Terminologia

$def(i)$ = insieme di variabili definite nel nodo i

$du(x, i) = \{j \mid x \in def(i), x \text{ usata in } j, \text{ cammino } i \rightarrow j \text{ libero da nuove definizioni di } x\}$

($du(x, i)$ è l'insieme dei nodi j dove x è definita in i e usata in j ecc..)

11.10 Criteri di copertura derivati

```
1      01 void main() {  
2          02 float a, b, x, y;  
3          03 read(x);  
4          04 read(y);  
5          05 a = x;  
6          06 b = y;  
7          07 while (a != b)  
8              08 if (a > b)  
9                  09 a = a - b;  
10             10 else  
11                 11 b = b - a;  
12             12 write(a);  
13         13 }
```

$du(5, a) = 7, 8, 9, 11, 12$ e $du(9, a) = 7, 8, 9, 11, 12$

1. **Copertura delle definizioni (Cdef):** ogni definizione testata fino a **almeno un uso** (libera da definizioni intermedie, quindi ogni cammino deve avere un d, poi prima di un'altra d una u).

a infatti viene definita in 5 e 9 e dobbiamo trovare dei test che le testino entrambe:

input: <8,4>

2. **Copertura degli usi (Cuse):** ogni definizione testata fino a **tutti gli usi**.

qui necessitiamo di 3 test:

input <4,8> <12,8> <12,4>

- il primo copre solo gli usi per a definita in 5 (d_5) di while e else (u_7, u_{11}, u_{12})
- il secondo solo gli usi di a definita in 9 (d_9) di while (u_7, u_9, u_{11}, u_{12})
Questo testa se a resta più grande di b anche dopo la sottrazione.

- il terzo serve per vedere come il valore di *a* modificato nel ciclo (d9) arrivi alla stampa finale (u12) passando per un giro in cui non viene toccato.
3. **Copertura dei cammini DU (CpathDU)**: ogni cammino da ogni definizione a ogni uso testato (impraticabile per combinatoria).

11.11 Oltre le variabili

Estensione a oggetti generici (es. file):

a: apertura (read/write)
l: lettura
s: scrittura
c: chiusura

Regole tipiche:

- l, s, c devono essere precedute da a senza chiusure intermedie.
- a deve essere seguita da c prima di una nuova apertura.

11.12 Criterio di copertura del budget

- Si creano casi di test finché non si esauriscono le risorse (tempo/soldi).
- Nessuna garanzia di efficacia → criterio **sconsigliato**.

12 Tecniche di Review

12.1 Introduzione

- Tecniche automatiche (test e analisi del codice) non rilevano errori dovuti a incomprensioni delle specifiche.
- Necessario intervento umano tramite **review** del codice.
- Spesso svolta da team esterno di testing → più imparzialità e maggiore efficacia.

12.2 Principali tecniche

1. Bebugging
2. Analisi mutazionale
3. Object Oriented Testing
4. Testing funzionale
5. Software inspection
6. Modelli statistici
7. Debugging

12.3 Bebugging

- Gli sviluppatori inseriscono deliberatamente n bug nel codice.
- Al team di testing viene comunicato il numero n di bug da trovare.
- Obiettivo: stimolare la ricerca e valutare l'efficacia del testing.
- Metrica: percentuale di bug trovati/ n .
- Limite: difficoltà nel calibrare la difficoltà dei bug artificiali.

12.4 Analisi Mutazionale

- Evoluzione del bebugging.
- Dato un programma P e un test T , si generano mutanti Π (varianti di P).
- Si esegue T su ogni mutante:
 - Se P è corretto, i mutanti devono produrre risultati diversi.
 - Misura la capacità del test di rilevare differenze, non la correttezza di P .

12.4.1 Criterio di copertura dei mutanti

- T soddisfa il criterio se ogni mutante $\pi \in \Pi$ produce almeno un output diverso da P .
- Metrica: frazione di mutanti rilevati rispetto al totale. (se un mutante non viene rilevato bisogna aggiungere test che lo rilevano)

12.4.2 Generazione dei mutanti

- Mutanti $\rightarrow$ differenze minime rispetto a P .
- Numero limitato (centinaia/migliaia) per motivi di tempo.
- Qualità dei mutanti determinata da **operatori mutanti**.

Operatori mutanti

- Modifiche su:
 - Costanti/variabili (es. scambio di valori).
 - Operatori/espressioni (es. $< \rightarrow \leq$).
 - Comandi (es. `while` $\rightarrow$ `if`).
- Specifici per domini:
 - Concorrenti (sincronizzazione).
 - Object-Oriented (interfacce e metodi).
- Tool automatici lavorano spesso sull'eseguibile dato che generare mutanti può essere troppo costoso computazionalmente(meno overhead).

12.4.3 High Order Mutation (HOM)

- Applica più modifiche su codice già mutato. (2 errori si possono mascherare a vicenda)
- Scopo: creare mutanti più difficili da rilevare → test più approfonditi.

12.4.4 Automazione

- L'obiettivo non è verificare la correttezza ma la **profondità dei test**.
- Non serve oracolo di correttezza (dato che non serve conoscere il funzionamento del programma) → maggiore automatizzabilità.
- Costo: $n + 1$ esecuzioni per n mutanti (+1 programma vero).
- Problemi:
 - Ripetizione di test casuali. (per generare il test fino a quando non copro il mutante)
 - Mutanti equivalenti (stessa semantica).
 - Timeout necessari per evitare loop infiniti.

12.5 Object-Oriented Testing

- Nei linguaggi OO l'unità testabile è la **classe**, non la funzione.
- I metodi dipendono dallo stato dell'oggetto.

12.5.1 Ereditarietà e collegamento dinamico

- Metodi ereditati vanno ritestati nelle sottoclassi.
- Non si eredita il testing, ma si possono ereditare i **casi di test**.
- Il collegamento dinamico complica la copertura (metodi risolti a runtime).

12.5.2 Testing di una classe

- Isolare la classe con *stub*.
- Implementare metodi mancanti.
- Aggiungere funzioni per accedere allo stato.

- Usare una classe *driver* per testare i metodi secondo un criterio di copertura.

12.5.3 Criteri di copertura per classi

- Basati su una **macchina a stati**:
 1. Copertura dei nodi.
 2. Copertura degli archi.
 3. Copertura di coppie archi input/output. (se ho 2 archi entranti e 2 uscenti devo avere 4 test uno per ogni combinazione)
 4. Copertura di tutti i cammini (spesso infiniti, impraticabile).

Tipo di testing

- Se la macchina a stati è parte delle specifiche → **black box**.
- Se è derivata dal codice → **white box**.

12.6 Testing Funzionale

- Verifica il comportamento esterno del programma (black box).
- Basato sulle **specifiche**, non sull'implementazione.
- Identifica errori come:
 - Funzionalità mancanti.
 - Errori di interfaccia o prestazioni.

12.6.1 Tecniche principali

1. Basate su grafi. (diagrammi di sequenza ecc.)
2. Suddivisione in classi di equivalenza. (suddivisione dei valori di dominio che causano lo stesso comportamento in modo da testare i vari comportamenti)
3. Analisi dei valori limite a cavallo delle suddivisioni (test di frontiera).
4. Collaudo per confronto (V nuova vs V vecchia cercando di non avere una regressione).

12.6.2 Testing delle interfacce

- Verifica comunicazione tra componenti.
- Tipi di interfacce:
 - A parametri, a memoria condivisa, a metodi sincroni, a messaggi.
- Errori comuni: parametri errati, precondizioni sbagliate, problemi di sincronizzazione.

12.6.3 Classi di equivalenza

- Domini di input suddivisi in insiemi di valori che producono lo stesso comportamento.
- Tipi:
 - Input validi.
 - Input non validi.
- Obiettivo: testare tutte le classi di comportamento, non tutti i valori. (es: uno per gli input validi e uno per i non validi se si parla di un pin del bancomat, non tutti i non validi).
- un altro esempio è valori corretti (da 200 a 400), classi di eq non corrette possono essere $n < 200$ e $n > 400$

12.6.4 Test di frontiera

- Si testano i valori al confine tra le classi di equivalenza.
- Gli errori si manifestano più spesso ai limiti.

12.6.5 Category Partition

- Metodo per derivare le classi di equivalenza dalle specifiche.
- Passi principali:
 1. Analisi delle specifiche e identificazione categorie.
 2. Scelta dei valori significativi.
 3. Definizione di vincoli tra categorie.
 4. Scrittura dei test. (generalmente seguendo il pattern 0, 1, 2 es: se il pattern richiede uno spazio vuoto, allora testare con 0,1,2 spazi vuoti)

Alpha e Beta Testing

- **Alpha:** test in-house (specifiche note).
- **Beta:** test su ambienti reali (diverse configurazioni).

Vincoli

- **Pairwise testing:** test di tutte le coppie di valori significativi imponendo vincoli e strutture logiche per scegliere i test.
- Tipi di vincoli:
 - *se*: se abbiamo definito una proprietà è inutile non rispettarla.
 - *single*: inutile avere più test che fanno la stessa cosa (se cerco una parola non mi interessa se la parola cambia se nessun test ha più di una occorrenza della parola cercata).
 - *error*: genera errore immediato (manca un parametro, grazie al cazzo che genera errore se il parametro serve).

12.6.6 Conclusioni

- Difficile individuare tutti i casi significativi.
- Necessario un oracolo per conoscere i risultati attesi.

13 Object Orientation e Testing Funzionale

- Il testing funzionale è **black box**: non dipende dai dettagli implementativi.
- In OO non cambia il testing funzionale a livello di *unità*, ma cambia il **testing di integrazione**.
- Nei linguaggi procedurali: test di integrazione integrando o con logica **bottom-up** o **top-down**.
- In OO: relazioni tra classi **non gerarchiche** (associazioni, aggregazioni, dipendenze), rendono difficile individuare i sottoinsiemi da testare.

Strumenti utili

- **Use case diagram** e scenari → test dei componenti coinvolti.
- **Sequence diagram** → test delle classi protagoniste nelle interazioni.
- **State diagram** → uso analogo a quello già descritto.

14 Software Inspection

- Tecniche manuali di verifica basate su analisi di gruppo.
- Esempio: **pair programming**.
- Nessun tool necessario, alta flessibilità.
- Può essere applicata a codice, specifiche o eseguibili in qualsiasi fase del ciclo di vita.

14.1 Fagan Code Inspection

- Metodo sviluppato da **Michael Fagan (IBM, anni '70)**.
- Gruppo di esperti che verifica il codice per individuare errori o incongruenze.

Ruoli principali

- **Moderatore**: coordina, seleziona i partecipanti, evita conflitti di interesse.
- **Reader/Tester**: leggono e interpretano il codice, ricercano difetti.
- **Autore**: risponde a domande, non corregge durante l'ispezione.

Processo

1. **Planning**: definizione ruoli, tempi, incontri.
2. **Overview**: contesto (idea iniziale) e condivisione dei materiali.
3. **Preparation**: comprensione individuale del codice.
4. **Inspection**: analisi collettiva, individuazione difetti.
5. **Rework**: correzione da parte dell'autore.
6. **Follow-up**: eventuale nuova ispezione.

Durante l'ispezione

- Si registrano i difetti, **non si correggono**.
- Max 2 sessioni/giorno, 2h ciascuna, circa 150 linee/ora.
- Possibile “test a mano” (testando e ripercorrendo le linee a mano) e uso di **checklist**.

14.1.1 Checklist

- Partono dagli errori comuni e servono a prevenirli.
- Possono essere interne all'azienda, evolutive col progetto.
- **Esempio NASA (1993)**: checklist per C e Fortran divise in *functionality, data usage, control, linkage, computation, maintenance, clarity*.

Esempi di punti checklist

- Ogni modulo ha una singola funzione?
- Il codice rispetta il design dettagliato?
- Tutte le costanti sono in maiuscolo?
- Pochi `#include` annidati? Commenti sufficienti?

Struttura di incentivi

- Difetti trovati **non usati per valutare gli autori**.
- Readers e tester incentivati in base ai difetti trovati.

Variante: Active Design Reviews introdotta per evitare il rischio che i reviewers si distraggano

- L'autore solleva questioni e interroga i revisori.
- Aumenta il coinvolgimento dei partecipanti.

14.2 Automazione

- Tool di supporto: formattazione, checklist elettroniche, navigazione codice, comunicazioni al team, controllo processi come git merge.

14.3 Pro e Contro

Vantaggi

- Processo rigoroso e auto-migliorante.
- Basato sull'esperienza (checklist).
- Incentivi sociali e mentali positivi.
- Analisi più astratta rispetto al testing.
- Applicabile a programmi incompleti.

Svantaggi

- Applicabile solo a livello di unità. (troppi dettagli sono difficili da comprendere)
- Non incrementale: follow-up (dopo la correzione una nuova revisione) spesso inefficace.

Legge di Fagan (L17)

Le ispezioni aumentano in maniera significativa la produttività, qualità e stabilità del progetto.

15 Confronto tra Tecniche di Verifica e Convalida

- Tecniche: testing strutturale, funzionale, ispezione.
- Efficacia variabile a seconda del progetto.
- Diverse tecniche trovano errori diversi.
- Nessuna tecnica è sempre la migliore.

15.1 Combinare le Tecniche

- Applicare più tecniche, anche superficialmente, è più efficace di usarne una sola in profondità.

Legge di Hetzel-Meyer (L20)

Una combinazione di diversi metodi di V/C supera qualsiasi metodo singolo.

16 Gruppi di Test Autonomi

Legge di Weinberg (L23)

Uno sviluppatore non è adatto a testare il suo codice.
per questo si preferisce di solito gruppi di tester autonomi

Vantaggi

- Specializzazione tecnica, conoscenza dei tool.
- Distacco dal codice e indipendenza psicologica nella valutazione (se testassi il mio codice sarei incentivato a non trovare errori).

Svantaggi

- Perdita di competenze di sviluppo.
- Maggiore distanza dal progetto e dai requisiti.
- Spostamento della responsabilità verso i tester (uno sviluppatore è meno attento perchè tanto c'è il tester).

Alternative

- **Rotazione personale:** alternare ruoli di dev/tester → meno pressione, meno specializzazione e più equilibrio.
- **Condivisione personale:** dev = tester (stesso gruppo che fa tutti e due) → conoscenza profonda del codice ma ritorno delle criticità viste prima.

17 Modelli Statistici

- Analizzano correlazioni tra metriche del codice (lunghezza, indentazione, linguaggio) e numero/tipo di errori.
- Possono suggerire dove testare di più, ma **non considerano le correzioni già fatte.**

Legge di Pareto/Zipf (L24)

Circa l'80% dei difetti proviene dal 20% dei moduli. (se una zona ha un errore è probabile che ne abbia più di uno)

18 Debugging

- Mira a localizzare e rimuovere le anomalie causa di malfunzionamenti noti (non per trovare i malfunzionamenti, quello ci pensano i test).
- Si basa sulla riproducibilità del malfunzionamento (se esiste davvero e non è dato da errori di specifica).

Difficoltà

- Relazione anomalia-malfunzionamento non biunivoca (magari un malfunzionamento è dato da più anomalie e non una sola).
- Correzioni possono introdurre nuovi errori.

Citazione di Kernighan

Debugging is twice as hard as writing the code in the first place.

18.1 Tecniche di Debugging

- **Naïve**: print di variabili → modifica codice, poco flessibile.
- **Naïve avanzata**: uso di `#ifdef`, logging, asserzioni.
- **Dump di memoria**: immagine completa della memoria (core dump), difficile da interpretare.
- **Debugging simbolico**: tool interattivi, variabili osservabili, *break-point*, *watch*, *spy monitor* (non alterano il codice aggiungendo righe ma osservano il funzionamento di esso alterando l'esecuzione (passo a passo), serve a vedere il valore di variabili in un determinato momento).
- **Debugging per prova**: watch condizionali (dici di controllare che una var sia uguale ad un'altra) e asserzioni dinamiche (verificano che alcune proprietà rimangano valide durante l'esecuzione del programma). Contiene tutte le funzionalità del debugging simbolico ma verifica anche le proprietà logiche

Funzionalità aggiuntive dei debugger i debugger possono anche:

- Modificare variabili o codice a runtime.
- Rappresentazioni grafiche di strutture dati dinamiche.

18.2 Automazione del Debugging

- **Andreas Zeller, Debugging Book:** introduce delle possibili spunti per un futuro con debugger automatici
 - **Shrinking input:** riduzione automatica dell'input problematico.
 - **Differential debugging:** variazioni minime del codice per identificare l'anomalia.
- **Git bisect:** ricerca dicotomica della versione che introduce il bug (confronta con la versione vecchia).